Navodya Danadhari

IT22082442_DRAFT

RP Final Report

Document Details

Submission ID
trn:oid:::3618:154324275

Submission Date
Oct 11, 2026, 6:23 PM GMT+5:30

Download Date
Oct 11, 2026, 6:30 PM GMT+5:30

File Name
IT22082442_DRAFT.pdf

File Size
1.9 MB

109 Pages

27,463 Words

168,246 Characters

A NEXT-GENERATION AI FRAMEWORK FOR INTELLIGENT TRAVEL SUPPORT IN SRI LANKA

INDIVIDUAL THESIS

T.M. Navodya Danadhari Lakmali Senevirathna
(IT22082442)

AI Conversational Travel Analyzer with Intelligent Package Generation and Real-Time Adaptive Engine

Group ID: R26-IT-112

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) Degree in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

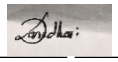
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (see Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Senevirathna T.M.N.D.L

Student ID: IT22082442

Signature:  Date: 10/11/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr.Kaushalya Dissanayake		
Co-supervisor: Ms.Suriyaa Kumari		

ABSTRACT

When people are planning a trip they have to organize a number of different aspects such as where they are going, the transport they will take, their accommodation, the activities they want to do, their budget, and how long the trip will last. Currently, most travel planning systems ask users to search on their own and then put together information from a variety of different services, and although many conversational systems give out reactive answers, they do not continuously adjust the itinerary in response to changing travel conditions. The research carried out in this project overcomes these shortcomings by developing an AI Conversational Travel Analyzer with Intelligent Package Generation and a Real-Time Adaptive Engine as part of the R26-IT-112 framework.

The system makes use of natural language processing in a conversational format in order to obtain the travel needs from user interactions. Travel information is extracted from prompts using Google Gemini, which is accessed via ChatGoogleGenerativeAI, and is then converted into a structured form. The process of generating packages involves assessing a number of possible routes on the basis of a number of criteria such as distance, travel time, road risk, weather risk, crowd pressure, attraction value, and lodging value. To determine appropriate travel packages, a multi-objective ranking method like NSGA-II is applied, this involving non-dominated sorting, crowding distance, and a normalized weighted compromise scoring method.

The system also includes real-time travel information such as weather conditions, traffic situations, road conditions, crowd data, and related travel warnings. If relevant conditions change, it can update the travel information and revise the package that has been generated. This allows the component to go beyond the creation of static itineraries and instead provide adaptive travel planning.

The focus of the individual contribution therefore lies in the extraction of conversational travel requirements, the optimisation of a multi-objective package, the

integration of real-time travel intelligence, the adaptation of the itinerary, and the incorporation of these capabilities into the broader R26-IT-112 framework.

Keywords: Conversational AI, Travel Optimization, NSGA-II, Natural Language Processing, Real-Time Adaptation

ACKNOWLEDGEMENTS

I sincerely thank my supervisor, Dr.Kaushalya Dissanayake, and my co-supervisor, Ms.Suriyaa Kumari, for their continuous guidance, constructive feedback and encouragement throughout this research. Their direction helped me to refine the research problem, the methodology and the structure of this thesis.

I am grateful to my group members (IT22216014, IT22176974 and IT22607614) for their collaboration and for the discussions that shaped the integration of the individual components of our group project.

I also thank the Department of Information Technology at the Sri Lanka Institute of Information Technology (SLIIT) for the academic resources and environment that made this work possible.

TABLE OF CONTENTS

Declaration of the Candidate and Supervisor i
Abstract ii
Acknowledgements iii
Table of Contents iv
List of Figures vii
List of Tables viii
List of Abbreviations x
List of Appendices xi
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 3
1.3 Problem Statement 3
1.4 Research Gap 4
1.5 Aim and Objectives 5
1.5.1 Aim 5
1.5.2 Specific objectives 5
1.6 Research Questions and Hypotheses 6
1.7 Scope, Limitations and Assumptions 6
1.7.1 Scope 6
1.7.2 Limitations and assumptions 6
1.8 Individual Contribution 7
1.9 Thesis Structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review Method and Scope 9
2.2 Domain and Theoretical Foundations 10
2.2.1 Smart tourism and information technology 10
2.2.2 Recommender systems 10
2.2.3 The tourist trip design problem 11
2.2.4 Multi-objective optimisation and Pareto efficiency 11
2.2.5 NSGA-II 12
2.2.6 Large language models for language understanding 12
2.2.7 Agents, tools and retrieval 13

2.2.8 Dynamic optimisation and re-planning 13
2.2.9 Evaluating multi-objective optimisers 14
2.2.10 Voice interfaces and proactive notification 14
2.3 Related Research 15
2.3.1 AI-supported travel planning 15
2.3.2 Conversational interfaces in tourism 15
2.3.3 Optimisation-based itinerary generation 16
2.3.4 Region-specific travel-time modelling 16
2.3.5 Disruption awareness 16
2.3.6 Summary of the related research 16
2.4 Existing Systems, Products and Techniques 17
2.5 Gap Synthesis and Conceptual Framework 19
3 RESEARCH METHODOLOGY 21
3.1 Research Design 21
3.1.1 Design Science Research 21
3.1.2 Research strategy 22
3.1.3 Evaluation design principles 22
3.2 Data, Participants and Study Materials 23
3.2.1 Travel knowledge base 23
3.2.2 Simulated planning scenarios 24
3.2.3 Annotated utterance set for the NLP Analyzer 25
3.2.4 Disruption scenarios for the Adaptive Engine 26
3.2.5 Participants for the usability study 26
3.3 Experimental and Study Procedure 27
3.3.1 E1: constraint extraction 27
3.3.2 E2: travel-time model 28
3.3.3 E3 and E4: optimisation quality and feasibility 28
3.3.4 E5: risk detection and re-optimisation 28
3.3.5 E6: response time 29
3.3.6 E7: usability study 29
3.4 Measures and Evaluation Metrics 29
3.4.1 Extraction metrics 29
3.4.2 Travel-time error 29
3.4.3 Optimisation metrics 30
3.4.4 Risk-detection metrics 30

3.4.5 Performance and usability metrics 30
3.4.6 Sensitivity of the risk threshold and algorithm parameters 30
3.5 Data Analysis 31
3.5.1 Experiment schedule and dependencies 31
3.6 Ethics, Privacy, Security and Responsible AI 32
3.6.1 Data protection 32
3.6.2 Responsible use of language models 32
3.6.3 Security 33
3.7 Reproducibility and Research Artefacts 33
4 PRODUCT AND SOLUTION DESIGN 34
4.1 Requirements 34
4.1.1 Stakeholders 34
4.1.2 Functional requirements 34
4.1.3 Non-functional requirements 35
4.2 Component Architecture within the Complete Solution 36
4.2.1 Overview 36
4.2.2 Responsibilities of the agents 37
4.2.3 Two operating phases 37
4.2.4 Integration with the rest of the group solution 38
4.2.5 Deployment view 38
4.3 Design Alternatives and Decisions 39
4.4 Detailed Design 41
4.4.1 Multi-Intent NLP Analyzer 41
4.4.2 Intelligent Package Generator 42
4.4.2.1 Problem formulation 42
4.4.2.2 Encoding 42
4.4.2.3 Objectives 42
4.4.2.4 Constraints 43
4.4.2.5 Terrain-aware travel-time model 43
4.4.2.6 Evolutionary operators and algorithm 44
4.4.2.7 Choosing the recommended package 45
4.4.2.8 Worked example of Pareto selection 46
4.4.2.9 Feasibility of the reference trip 47
4.4.3 Real-Time Adaptive Engine (Guardian Loop) 47
4.4.3.1 Signals and risk factors 47

4.4.3.2 Segment risk score 48
4.4.3.3 Trigger rule 49
4.4.3.4 Warm-start re-optimisation 49
4.4.4 Proactive Voice Agent 51
4.4.5 Data model 51
4.4.6 User interface design 52
4.4.7 Failure modes and graceful degradation 53
4.4.8 Threat model 53
4.5 Standards and Professional Practice 54
5 IMPLEMENTATION, INTEGRATION AND TESTING 55
5.1 Development Environment and Technology Stack 55
5.2 Component Implementation 55
5.2.1 NLP Analyzer 55
5.2.2 Package Generator 56
5.2.3 Adaptive Engine 57
5.2.4 Voice Agent 58
5.2.5 Mobile client 58
5.3 Integration with the Other Components 58
5.4 Verification and Software Testing 59
5.4.1 Test strategy 59
5.4.2 Test plan 60
5.4.3 Interface contract 61
5.5 Deployment and Operation 61
6 RESULTS AND VALIDATION 62
6.1 Research Results 62
6.2 Product and Technical Results 62
6.2.1 E6: response time 62
6.2.2 Requirements coverage 63
6.3 User and Stakeholder Validation 64
6.3.1 Interpreting the usability score 65
6.4 Integration Results 66
6.5 Summary of Findings 66
7 DISCUSSION 68
7.1 Interpretation of Findings 68
7.2 Comparison with Literature Baselines and Existing Solutions 68

7.3 Research Contribution 69
7.4 Product Contribution 70
7.5 Limitations and Threats to Validity 70
7.5.1 Construct validity 70
7.5.2 Internal validity 70
7.5.3 External validity 70
7.5.4 Conclusion validity 71
7.5.5 Safety and ethical limitations 71
7.5.6 Mitigations and residual risk 71
7.6 Implications 72
8 COMMERCIALISATION AND SUSTAINABILITY 73
8.1 Value Proposition and Target Users 73
8.1.1 Value proposition 73
8.1.2 Target users 73
8.2 Market and Competitive Position 74
8.2.1 Market context 74
8.2.2 Competitive positioning 74
8.3 Business and Sustainability Model 75
8.3.1 Revenue streams 75
8.3.2 Cost structure and project budget 75
8.3.3 Go-to-market 76
8.3.4 Sustainability of the model 77
8.3.5 Pricing tiers and unit economics (assumptions) 77
8.4 Scalability, Intellectual Property and Compliance 78
8.4.1 Scalability 78
8.4.2 Intellectual property 78
8.4.3 Legal and regulatory compliance 78
8.5 Contribution to the Sustainable Development Goals 79
9 CONCLUSION AND FUTURE DIRECTIONS 80
9.1 Conclusion 80
9.2 Achievement of Objectives 80
9.3 Individual Research and Product Contributions 81
9.4 Future Directions 81
REFERENCES 83
GLOSSARY 86

APPENDICES 87
Appendix A: Research Instruments and Extended Material 87
A.1 Annotation guideline and example utterances 87
A.2 Extraction prompt template 87
A.3 System Usability Scale items 87
A.4 Participant information and consent (summary) 88
Appendix B: Academic Integrity and AI Use Disclosure 88
Appendix C: Risk Analysis 89
Appendix D: Sample Travel Dataset Structure 89
Appendix E: User Interface Mock-ups 90
Appendix F: Example Service Payloads 91
F.1 Constraint object returned by the NLP Analyzer 91
F.2 Package response returned by the Package Generator 91
F.3 Change event passed to the Voice Agent 92
Appendix G: Project Plan and Work Breakdown 92
Appendix H: Additional Test Case Matrix 93

LIST OF FIGURES

Figure 1.1: Reactive assistance compared with the proposed proactive approach 2

Figure 2.1: Landscape of related work and the position of the proposed system 19

Figure 3.1: Design Science Research process applied in this study, adapted from the process model of Peffers et al. 21

Figure 3.2: Overview of the experimental design: inputs, systems under test, metrics and analysis 23

Figure 4.1: Architecture of the Tri-Agent system and its data stores 37

Figure 4.2: Deployment view of the system on a cloud container platform 39

Figure 4.3: Chromosome encoding of a travel package..... 42

Figure 4.4: NSGA-II procedure used by the Intelligent Package Generator 45

Figure 4.5: Risk scoring pipeline of the Real-Time Adaptive Engine..... 49

Figure 4.6: The Guardian Loop: monitoring, risk scoring, re-optimisation and notification 50

Figure 4.7: Principal entities of the itinerary and state store and their relationships 52

LIST OF TABLES

Table 1.1: Research questions and hypotheses 6
Table 2.1: Review themes and representative search terms 9
Table 2.2: Capability comparison of representative approaches 17
Table 2.3: Capability comparison of categories of travel-planning system 18
Table 2.4: Mapping of research gaps to design response and onjectives 20
Table 3.1: Application of the DSR activities in this thesis 22
Table 3.2: Travel knowledge base entitie and sources 24
Table 3.3: Scenario factors and levels 25
Table 3.4: Experiments, hypotheses and baselines 27
Table 3.5: Initial NSGA-II settings 28
Table 3.6: Order and dependencies of the experiments 32
Table 4.1: Functional requirements 34
Table 4.2: Non-functional requirements and measurable targets 35
Table 4.3: Latency budget allocated to each stage (design targets) 36
Table 4.4: Responsibilities and interfaces of the three agents 37
Table 4.5: Design alternatives and decisions 40
Table 4.6: Initial parameters of the terrain-aware travel-time model 44
Table 4.7: Illustrative candidate packages for the worked example (not experimental data) 46
Table 4.8: Risk factors and normalization functions 48
Table 4.9: Initial weights of risk factors by segment type 48
Table 4.10: Principal entities of the itinerary and state store 51
Table 4.11: Failure modes and designed responses 53
Table 4.12: Principal threats and design controls 53
Table 5.1: Technology stack 55
Table 5.2: Principal service interfaces 58
Table 5.3: Representative test cases 60
Table 5.4: Principal service endpoints 61
Table 6.1: Evaluation Results of the Proposed System 62
Table 6.9: Stage-wise response time (seconds) 62
Table 6.10: Status of functional and non-functional requirements 63
Table 6.11: Usability study results 64
Table 6.12: Summary of hypothesis outcomes 66
Table 8.1: Target user segments 73

Table 8.2: SWOT analysis 74
Table 8.3: Estimated project budget..... 75
Table 8.4: Proposed pricing tiers (hypotheses to be validated)..... 77
Table 8.5: Contribution to the Sustainable Development Goals..... 79
Table 9.2: Achievement of the specific objectives 80

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DSR	Design Science Research
LLM	Large Language Model
NLP	Natural Language Processing
NSGA-II	Non-dominated Sorting Genetic Algorithm II
UI	User Interface
UX	User Experience
WBS	Work Breakdown Structure

LIST OF APPENDICES

Appendix A: Research Instruments and Extended Material 85
Appendix B: Academic Integrity and AI Use Disclosure 86
Appendix C: Risk Analysis 87
Appendix D: Sample Travel Dataset Structure 87
Appendix E: User Interface Mock-ups 88
Appendix F: Example Service Payloads 88
Appendix G: Project Plan and Work Breakdown 89
Appendix H: Additional Test Case Matrix 90

1 INTRODUCTION

1.1 Overall Project Context

International tourism is one of the largest and fastest-recovering service sectors in the world. UN Tourism estimates that about 1.52 billion international tourist arrivals were recorded globally in 2025, roughly four per cent more than in 2024 [1]. Sri Lanka has shared in this recovery: the country recorded more than 2.36 million arrivals in 2025, the highest annual figure in its history, surpassing the previous record of 2,333,796 set in 2018, with 2,053,465 arrivals in 2024 [2], [3]. Official targets envisage more than five million visitors a year by 2030 [2].

Growth of this kind increases pressure on the tools that travellers use to plan their journeys. A typical visitor to Sri Lanka does not stay in one place. A four-day trip may combine the cultural capital of Kandy, the hill-country town of Nuwara Eliya or Ella, and the southern coast around Galle. Each leg has its own transport options, travel times and risks, and each must be reconciled with a flight, accommodation and a fixed budget. The traveller must assemble this plan manually from flight search engines, hotel aggregators, map applications and review sites, none of which reason about the plan as a whole.

The group project of which this thesis is a part, titled *Next-Generation AI Framework for Intelligent Travel Support in Sri Lanka*, addresses this problem with a set of cooperating intelligent components. The other group members contribute three complementary components to the integrated smart tourism framework: IT22216014 focuses on emotion recognition and tourist experience personalization, IT22176974 focuses on driver and vehicle safety, and IT22607614 focuses on crowd management and flow optimization.

This thesis reports the individual component of the author (IT22082442): an end-to-end pipeline that understands a free-text travel request, generates an optimised travel package, and then monitors and adapts that package proactively while the trip is under way.

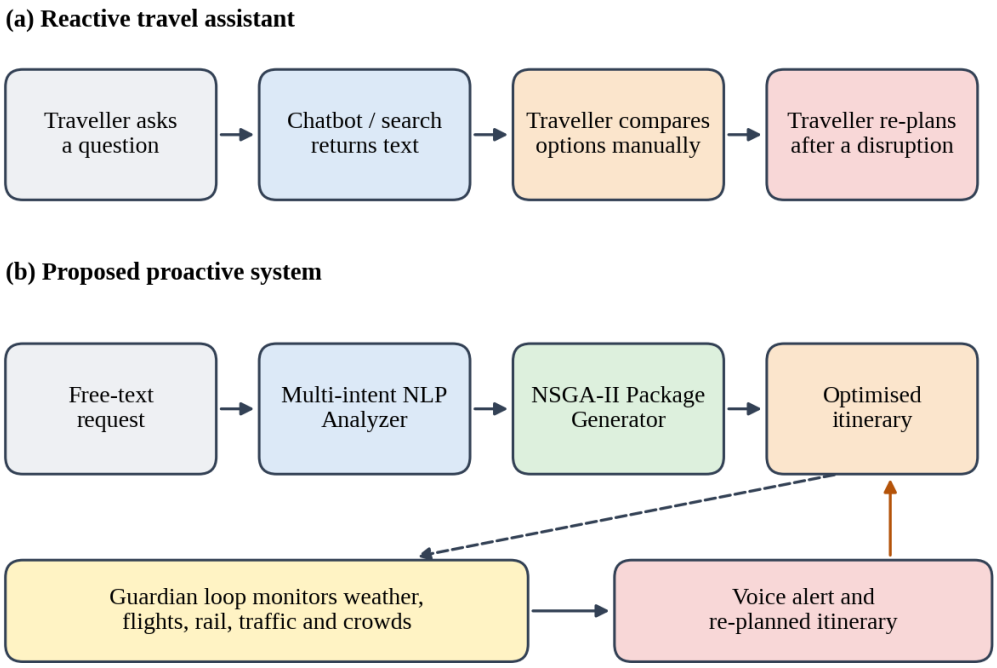


Figure 1.1: Reactive assistance compared with the proposed proactive approach

Figure 1.1 contrasts the two interaction models. In the reactive model, every step after the first question depends on the traveller: the traveller reads the answer, compares options manually, and re-plans when something goes wrong. In the proactive model studied here, the system converts the request into constraints, generates a package by optimisation, and then keeps watching the conditions that could invalidate the package. The traveler is contacted only when a change matters. This shift from answering to watching is the central idea of the thesis and shapes every design decision in Chapters 3 and 4.

The shift also changes what counts as success. A reactive assistant is judged by the quality of a single answer. A proactive assistant must be judged over time: whether it detects genuine problems early, whether it avoids false alarms that would train the traveller to ignore them, and whether the repaired plan is close enough to the original that the traveller can accept it without starting again. These considerations motivate the evaluation measures described in Section 3.4, including detection delay, alert precision and plan stability, which have no counterpart in a conventional chatbot evaluation.

1.2 Individual Component Context

The individual component is the **AI Conversational Travel Analyzer with Intelligent Package Generation and Real-Time Adaptive Engine**. It is organised as a Tri-Agent architecture in which three specialised agents cooperate through a shared itinerary state:

- **Multi-Intent NLP Analyzer.** A large language model (LLM) interprets a natural-language request, identifies the intents it contains, and extracts structured constraints such as budget, duration, destinations, transport preferences and travel style.
- **Intelligent Package Generator.** A constrained multi-objective optimiser, the Non-dominated Sorting Genetic Algorithm II (NSGA-II) [4], selects flights, hotels, inter-city transport and activities that minimise cost and travel time and maximise comfort and preference fit within the stated budget.
- **Real-Time Adaptive Engine with Proactive Voice Agent.** A monitoring loop, referred to in this thesis as the Guardian Loop, tracks weather, flight status, rail and road conditions and crowd levels, computes a risk score for each remaining segment of the trip, re-optimises the plan when the score exceeds a threshold, and informs the traveller by voice before the disruption affects the trip.

The central idea is a change of posture. Conventional travel assistants are reactive: they respond when the user asks. The system studied here is proactive: it treats a travel package as a living plan that must remain valid as conditions change, and it takes the initiative to repair the plan.

1.3 Problem Statement

Planning a multi-destination trip within a fixed budget is a combinatorial decision problem with competing objectives. Choosing a cheaper flight may force a longer or less comfortable overland journey; choosing a better hotel may consume the budget needed for transport. Existing planning tools expose this complexity to the traveller rather than resolving it. Three specific problems motivate this research.

- **From Information Overload to Guided Conversations.** Instead of confusing chatbots with messy paragraphs containing flights, dates, budgets, and cities all at once, we need a guided, step-by-step chat system that breaks the planning process into clear, manageable stages like flight booking and trip planning.
- **Lack of true optimisation.** Generative AI assistants can write plausible itineraries, but plain text generation does not guarantee that a plan satisfies a numeric budget, that travel times are feasible, or that the result is Pareto-efficient with respect to cost, time and comfort [5].
- **Static plans in a dynamic environment.** Sri Lankan travel is exposed to monsoon rainfall, landslide-prone mountain roads, rail delays and seasonal crowding. The plan that was optimal at booking time may be infeasible or unsafe a day later, yet most tools do not monitor conditions or repair the plan automatically.

The problem addressed by this thesis is therefore: *how can a conversational AI system convert an unstructured travel request into an optimised, budget-feasible travel package that accounts for Sri Lankan travel conditions, and keep that package valid by detecting disruptions and adapting the plan proactively?*

1.4 Research Gap

The literature review in Chapter 2 identifies four gaps that remain open despite substantial progress in tourism recommender systems, tourist trip design and LLM-based planning.

1. **Reactive versus proactive behaviour.** Tourism chatbots and recommender systems respond to queries but seldom monitor travel conditions or initiate plan changes [6], [7].
2. **Generation without optimisation.** Language-model planners are evaluated mainly on whether a plan is coherent, and benchmarks show that they frequently violate explicit constraints such as budget [5]. Classical tourist trip design algorithms optimise rigorously but do not accept natural-language input [8], [9].

3. **Weak regional modelling.** Global planners generally assume uniform travel speeds. In Sri Lanka, travel time depends strongly on road class, terrain and mode, so a distance-based estimate can mislead.
4. **Fragmented capability.** Natural-language understanding, multi-objective package optimisation and real-time adaptive monitoring are usually studied separately. Few systems integrate all three in one architecture and evaluate the integrated behaviour.

1.5 Aim and Objectives

1.5.1 Aim

To design, implement and evaluate an AI-based proactive travel planning system that generates optimised, personalised vacation packages for Sri Lanka within a fixed budget and automatically adapts them to real-time environmental and transport disruptions.

1.5.2 Specific objectives

1. To develop a Multi-Intent NLP Analyzer that extracts structured travel constraints from free-text input and asks for clarification when required fields are missing.
2. To formulate package generation as a constrained multi-objective optimisation problem and solve it with NSGA-II under a total-cost constraint.
3. To model Sri Lanka-specific travel-time variation caused by road class, terrain and transport mode.
4. To design a Real-Time Adaptive Engine that monitors weather, flight status, rail and road conditions and crowd levels, computes a risk score, and re-optimises the itinerary when a threshold is exceeded.
5. To build a Proactive Voice Agent that notifies the traveller and proposes alternatives before a disruption affects the trip.
6. To evaluate the system using extraction accuracy, optimisation quality, budget feasibility, response time and risk-detection accuracy, and to compare it with defined baselines.

1.6 Research Questions and Hypotheses

The objectives are operationalised as four research questions, each paired with a testable hypothesis. The numeric thresholds are design-time success criteria, fixed before experiments are run; they are not reported results.

Table 1.1: Research questions and hypotheses

ID	Research question	Hypothesis and success criterion
RQ1	How accurately can an LLM with schema-constrained output extract travel constraints from free-text requests?	**H1.** Field-level micro-F1 of at least 0.90 on the annotated test set, and significantly higher than a rule-based baseline.
RQ2	Does NSGA-II produce better trade-offs than simpler search strategies under the same evaluation budget?	**H2.** NSGA-II attains a significantly larger hypervolume than random search, greedy construction and a weighted-sum genetic algorithm (Wilcoxon signed-rank, $p < 0.05$), with a constraint-violation rate of 0%.
RQ3	Does a terrain-aware travel-time model improve the realism of generated itineraries?	**H3.** The terrain-aware model has a lower mean absolute error against reference journey times than a constant-speed model.
RQ4	Can a risk-scoring Guardian Loop detect disruptions and repair the itinerary quickly enough to be useful?	**H4.** On injected-disruption scenarios, recall of at least 0.90 and precision of at least 0.80; median re-optimisation latency below 5 s.

1.7 Scope, Limitations and Assumptions

1.7.1 Scope

- The system targets inbound international travellers planning 3 to 10-day trips to Sri Lanka, with the Colombo, Kandy, Nuwara Eliya, Ella, Sigiriya, Galle and southern-coast corridor as the primary geographic scope.
- Inputs are English-language text or speech.
- The system produces and adapts itineraries; it does not execute payments or confirm bookings with suppliers.

1.7.2 Limitations and assumptions

- Flight and attraction prices are taken from publicly available datasets and snapshots, so absolute prices are indicative rather than live quotations.

- Real-time feeds depend on the availability of third-party APIs. Where a public rail feed is not available, the rail-status signal is simulated from published timetables and injected delay scenarios.
- Evaluation relies mainly on simulated scenarios and a small usability study. The findings therefore demonstrate feasibility and relative performance, not market-scale validity.
- Sinhala and Tamil input, including code-mixed text, is outside the evaluated scope and is treated as future work.

1.8 Individual Contribution

The author is responsible for the design, implementation and evaluation of all three agents and the voice alert pathway described in this thesis. The contributions are as follows.

- A schema-constrained LLM extraction pipeline with validation, normalisation of Sri Lankan monetary expressions, and a clarification dialogue.
- A constrained NSGA-II formulation of the Traveller Constraint Satisfaction Problem with a mixed discrete chromosome and a terrain-aware travel-time model.
- A Guardian Loop with a transparent, segment-level risk-scoring function and a warm-start, plan-stability-aware re-optimisation procedure.
- A Proactive Voice Agent that verbalises only verified structured changes to limit hallucination.
- An evaluation framework with baselines, metrics and statistical tests for the integrated system.

This thesis reports my individual component, the AI Conversational Travel Analyzer with Intelligent Package Generation and Real-Time Adaptive Engine (IT22082442). I implemented the NSGA-II-based route-ranking module (nsgaii/select_routes.py), the mobile client's conversational interface, API layer and voice-notification presentation components. The IoT device-monitoring and fleet-administration modules, and the

emotion check-in feature, were developed by other group members (IT22216014, IT22176974 and IT22607614) and are described only where they interface with my component. The Expo project scaffolding and navigation shell are shared.

1.9 Thesis Structure

Chapter 2 reviews the literature on smart tourism, tourist trip design, multi-objective evolutionary optimisation, LLM-based language understanding and planning, and dynamic re-planning. Chapter 3 describes the Design Science Research methodology, the data, the experimental procedure and the evaluation metrics. Chapter 4 presents the requirements, architecture and detailed design. Chapter 5 describes the implementation, integration and testing. Chapter 6 reports results and validation. Chapter 7 discusses the findings, contributions and threats to validity. Chapter 8 sets out the commercialisation and sustainability plan. Chapter 9 concludes the thesis and outlines future work.

2 LITERATURE REVIEW AND RELATED WORK

This chapter reviews the research on which the individual component is built. It begins with the review method, then covers the theoretical foundations of smart tourism, tourist trip design, multi-objective evolutionary optimisation, language-model-based understanding and planning, and dynamic re-planning. It then compares existing systems and synthesises the gaps that the thesis addresses.

2.1 Review Method and Scope

A structured narrative review was conducted. The aim was not to enumerate every tourism paper but to identify the strands of research that bear directly on the three agents of the system and to find where they have not been combined. Five themes guided the search, as summarised in Table 2.1.

Table 2.1: Review themes and representative search terms

Theme	Representative search terms	Relevance to the component
Smart tourism and AI	smart tourism; eTourism; AI in tourism; tourism recommender	Context; Agent 1 and Agent 3
Conversational understanding	intent detection; slot filling; LLM information extraction; structured output	Agent 1
Tourist trip design	tourist trip design problem; orienteering problem; itinerary optimisation	Agent 2
Multi-objective evolutionary optimisation	NSGA-II; Pareto; hypervolume; constraint handling	Agent 2
Dynamic re-planning and LLM agents	dynamic optimisation; re-optimisation; LLM agents; tool use; TravelPlanner	Agent 3 and voice agent

Source: Author.

To ground this research, we reviewed existing studies on smart tourism systems, AI-driven travel planning, and route optimization. We searched academic databases—specifically Google Scholar and IEEE Xplore—for papers published up to 2026 using keywords like "smart tourism," "travel itinerary planning," "LLM agents," and "multi-objective optimization." We focused on peer-reviewed, English-language papers that addressed travel planning, intent extraction, or optimization algorithms, while filtering out blog posts and duplicate records. Finally, we uncovered additional relevant studies by following the reference lists of our primary sources.

Foundational works were included regardless of age (for example the NSGA-II paper), while for fast-moving topics such as LLM agents the review favours recent peer-reviewed and widely cited sources. Commercial products are discussed in Section 2.4 on the basis of their publicly documented capabilities.

2.2 Domain and Theoretical Foundations

2.2.1 Smart tourism and information technology

Information technology has reshaped tourism over several decades. Buhalis and Law reviewed twenty years of research on tourism and showed how the Internet transformed distribution, marketing and the way travellers search for and book services [10]. Gretzel and colleagues later defined smart tourism as tourism supported by integrated technological infrastructure, including sensing, cloud services and artificial intelligence, that creates data-driven, personalised experiences [11]. Xiang and colleagues demonstrated that large volumes of traveller-generated text can be analysed to understand guest experience and satisfaction [12]. Tussyadiah reviewed research on automation in tourism and observed that AI and robotics are moving from information provision toward service delivery and decision support, while raising questions of trust and responsibility [6].

Together these works establish that digital intelligence is now central to tourism and that the research frontier has shifted from providing information to supporting decisions. The component studied here follows that direction by moving from recommending items to constructing and maintaining a complete, feasible plan.

2.2.2 Recommender systems

Recommender systems are the most common form of AI in tourism. They estimate the relevance of items such as destinations, hotels and attractions for a user, using collaborative filtering, content-based filtering, knowledge-based or hybrid methods [7]. Tourism recommendation is distinctive because items are consumed in sequence and under constraints of time, space and money, so the value of an item depends on what else is in the plan. A ranked list of attractions does not solve the problem of building a schedule. This observation connects recommendation to the trip design problem described next.

2.2.3 The tourist trip design problem

The tourist trip design problem (TTDP) asks which points of interest a tourist should visit, and in what order, to maximise the collected benefit within limited time and budget. It is commonly modelled as a variant of the orienteering problem, in which a path must be chosen through a set of scored nodes subject to a travel-length limit, and which is NP-hard [9]. Gavalas and colleagues surveyed algorithmic approaches to TTDP and its extensions, including time windows, multiple days, mandatory visits and multiple objectives, and concluded that heuristic and metaheuristic methods dominate practical solutions [8].

Most TTDP formulations concentrate on sequencing attractions within one city or region. They typically assume that the traveller has already arrived and that lodging and inter-city transport are given. The problem addressed in this thesis is broader in one respect, because the decision includes the flight, accommodation tier and inter-city transport mode, and narrower in another, because the benefit of individual attractions is represented at the level of daily activity sets rather than detailed minute-by-minute routing.

2.2.4 Multi-objective optimisation and Pareto efficiency

When objectives conflict, no single solution is best on all of them. A solution **x** is said to dominate **y**, written $x \prec y$, if x is no worse than y on every objective and strictly better on at least one. For a minimisation problem with objectives $f_1,\ldots,f_m$:

$$x \prec y \iff \forall i: f_i(x) \le f_i(y) \ \wedge \ \exists j: f_j(x) < f_j(y) \tag{2.1}$$

The set of non-dominated solutions is the Pareto front. Presenting the front, rather than a single weighted score, shows the traveller the cost, time and comfort trade-offs and avoids having to fix subjective weights in advance. Quality of an approximation to the front is commonly measured by the hypervolume, the volume of objective space dominated by the solution set and bounded by a reference point [13]. Hypervolume is attractive because it rewards both convergence to the front and diversity along it.

2.2.5 NSGA-II

NSGA-II is a widely used multi-objective evolutionary algorithm introduced by Deb and colleagues [4]. It addresses three weaknesses of earlier algorithms: the computational cost of non-dominated sorting, the lack of elitism, and the need for a user-specified sharing parameter. Its main elements are:

- **Fast non-dominated sorting.** The population is divided into fronts F_1, F_2, … by repeated removal of non-dominated individuals, with an overall complexity of $O(MN^2)$ for M objectives and population size N.
- **Crowding distance.** Within a front, each solution receives a diversity score based on the distance to its neighbours in objective space; boundary solutions receive infinite distance.
- **Crowded-comparison selection.** In binary tournaments, a solution with a better (lower) rank wins; when ranks are equal, the one with larger crowding distance wins.
- **Elitism.** Parent and offspring populations are merged and the next generation is filled front by front, so good solutions are never lost.
- **Constrained domination.** A feasible solution dominates an infeasible one; between two infeasible solutions the one with the smaller overall constraint violation dominates; between two feasible solutions ordinary domination applies. This rule lets the algorithm handle a hard budget constraint without penalty weights.

These properties suit the package-generation problem: the budget is a hard constraint, the objectives are conflicting, the decision variables are discrete, and the traveller benefits from seeing a diverse set of feasible alternatives. Open-source implementations such as DEAP [14] and pymoo [15] make the algorithm practical to implement and to compare with baselines.

2.2.6 Large language models for language understanding

Conventional natural-language understanding for dialogue systems treats intent detection and slot filling as supervised sequence tasks, often with a joint model. Chen

and colleagues showed that fine-tuning BERT [16] jointly for intent classification and slot filling improved accuracy over earlier models [17], and Weld and colleagues surveyed the broader family of joint models [18]. These approaches require labelled training data for each domain and slot schema.

Large language models change this trade-off. Brown and colleagues showed that a sufficiently large model can perform new tasks from a handful of examples placed in the prompt, without gradient updates [19]. Wei and colleagues showed that eliciting intermediate reasoning steps improves performance on multi-step problems [20]. Open-weight models such as the Llama 3 family [21] and proprietary models such as GPT-4o [22] can be prompted to return JSON that conforms to a declared schema, which makes them practical as information extractors for a domain like travel where labelled data is scarce.

The same flexibility introduces risk. Language models can produce fluent but unsupported content, commonly called hallucination [23]. For extraction this means that a model may invent a value that the user never stated, or silently convert a unit incorrectly. A system that feeds extracted values into an optimiser therefore needs a validation layer and the ability to ask the user to confirm uncertain fields.

2.2.7 Agents, tools and retrieval

An agent perceives its environment and acts to achieve goals, and a utility-based agent chooses actions that maximise expected utility [24]. In the LLM era, agentic designs combine a language model with external tools. React interleaves reasoning traces with tool calls so that a model can look up information and act on it [25]. Toolformer showed that models can learn when to call tools such as calculators or search engines [26], and retrieval-augmented generation grounds generation in retrieved documents [27]. These works motivate a design principle used in this thesis: let the language model handle language, and delegate arithmetic, search and constraint satisfaction to deterministic or algorithmic components that can be verified.

2.2.8 Dynamic optimisation and re-planning

In dynamic optimisation, the problem changes while it is being solved. Nguyen, Yang and Branke surveyed evolutionary approaches and identified recurring techniques:

detecting that a change has occurred, maintaining diversity, using memory of past solutions, and re-using the previous population to accelerate adaptation [28]. In vehicle routing, Pillac and colleagues distinguish between *re-optimisation* approaches, which re-solve the problem with updated data, and *policy* approaches that react by rule, and they note the importance of stability so that frequent small changes do not confuse operators [29]. Both observations transfer to itineraries: a traveller who is part-way through a trip needs a repair that preserves the completed portion and changes as little as necessary.

2.2.9 Evaluating multi-objective optimisers

Because a multi-objective optimiser returns a set of solutions rather than a single value, its quality cannot be summarised by one objective value. The literature distinguishes three properties of an approximation set: convergence, meaning closeness to the true Pareto front; spread, meaning coverage of the front; and uniformity of distribution [30]. No single unary indicator measures all three perfectly, but the hypervolume indicator, which measures the volume of objective space dominated by the set and bounded by a reference point, is widely used because it is strictly monotonic with respect to Pareto dominance [13], [30]. A set that dominates another set always has a larger hypervolume, which makes the indicator a safe basis for comparing methods.

Two practical rules apply to our evaluation. First, we must use a fixed reference point across all methods to keep the comparison fair. Second, because travel costs in rupees and travel times in hours are on completely different scales, we have to normalize these objectives before calculating the hypervolume (as explained in Section 3.4.3). As Coello and other experts point out, comparing random optimisers is tricky and requires careful performance checks, multiple runs, and non-parametric statistical tests. [31].

2.2.10 Voice interfaces and proactive notification

Speech technology has improved to the point where robust recognition across accents and noise conditions is available from general-purpose models trained on large amounts of weakly supervised audio [32]. For a travel assistant used on the move, this matters because the traveller may be walking, driving or carrying luggage, and a spoken alert can be received when a screen cannot be read. However, proactive

communication introduces a design problem that conversational systems rarely face: deciding when to interrupt. An alert that arrives too often is ignored, and an alert that arrives too late is useless. The design in Chapter 4 therefore treats alert generation as a controlled event with a threshold, a persistence requirement and a cooldown, rather than as a by-product of every data update.

A second design problem is truthfulness. Language models can generate fluent statements that are not supported by the data [23]. If a voice agent improvised its explanations, it could assert a cause for a disruption that the monitoring data does not show. For this reason the Voice Agent in this thesis is given a structured change set and is permitted only to verbalise that set. This constrains the model to restating facts that the Adaptive Engine has already established.

2.3 Related Research

2.3.1 AI-supported travel planning

Recent research applies language models directly to travel planning. The TravelPlanner benchmark evaluates agents on realistic multi-constraint planning tasks that require gathering information with tools and satisfying commonsense and user constraints, including budget [5]. Its results are instructive: even strong language-model agents achieved low success rates, with failures often caused by violations of hard constraints and by the difficulty of coordinating many pieces of information. The result supports a hybrid design in which an optimiser, rather than the language model alone, is responsible for constraint satisfaction.

2.3.2 Conversational interfaces in tourism

Tourism chatbots and virtual assistants answer questions, recommend places and sometimes support booking. A consistent limitation, noted in the reviews above, is that they are demand-driven and rely on the user to initiate each interaction [6]. They also tend to treat each request independently, without maintaining a global plan whose consistency must be preserved across turns. The proposed system instead maintains an explicit itinerary state that each agent reads and updates.

2.3.3 Optimisation-based itinerary generation

Optimisation-based work on itineraries uses integer programming, constraint programming and metaheuristics, with genetic algorithms, ant colony optimisation, iterated local search and simulated annealing frequently reported [8]. Multi-objective variants consider benefit, cost and time simultaneously. The literature generally supports the suitability of evolutionary algorithms when the search space is large, the objectives are non-linear and constraints are mixed. A remaining gap is the coupling of such optimisers to natural-language input and live data, so that the planning problem is built and re-built automatically from conversation.

2.3.4 Region-specific travel-time modelling

Calculating travel time is tricky because it depends on everything from road curves and traffic to weather and train delays. While global apps usually just give you a single-route estimate, they don't help optimize for cost or personal comfort. In Sri Lanka, assuming a uniform speed just doesn't work given the massive contrast between smooth expressways and winding hill-country roads. To solve this, our research uses a customized travel-time model broken down by road class, terrain, and travel mode, which we can test and calibrate against real-world journey times (as discussed in Chapter 3).

2.3.5 Disruption awareness

Disruption-aware travel research mostly addresses airline schedule recovery and urban public transport. For leisure itineraries, weather and safety alerts are generally provided as separate services that the traveller must consult. The impacts of Cyclone Ditwah in November 2025, which were reported among the factors affecting tourism that year [2], illustrate that a weather event can alter the feasibility of an entire itinerary; this motivates a monitor that evaluates risk at the level of each remaining segment and triggers a plan-level response.

2.3.6 Summary of the related research

Taken together, the research reviewed above shows maturity in each of the four strands but little integration across them. Recommender systems are well developed but select items rather than assembling constrained packages [7]. Conversational and language-

model approaches handle open-ended requests, but benchmark evidence shows that agents can struggle to satisfy several hard constraints at once when planning without a solver [5]. Tourist trip design research supplies mature optimisation formulations, but mainly for single-day routes between points of interest [9], [8]. Dynamic optimisation research supplies methods for handling change [28], [29], but these have rarely been connected to a conversational front end for travellers or tuned for the road and rail conditions of a particular country. The contribution of this thesis is therefore one of integration and adaptation, supported by rigorous evaluation, rather than the invention of a new algorithm.

2.4 Existing Systems, Products and Techniques

The capabilities of mainstream travel products are compared in Table 2.2 with the approach studied here. The comparison is qualitative, based on the publicly documented purpose of each product, and it is limited to the capabilities relevant to this thesis. It is not an evaluation of overall product quality.

Table 2.2: Capability comparison of representative approaches

Approach	Free-text input	Joint cost, time and comfort optimisation	Proactive disruption handling	Sri Lanka terrain model	Voice alerts
Map and navigation applications	Limited	No	Route-level traffic only	Generic	Navigation prompts
Review and booking platforms	Limited	No (filters and ranking)	Limited	No	No
General-purpose LLM chat assistants	Yes	No (text generation)	No (not monitoring)	Only if prompted	Optional
Tourist trip design research prototypes	No	Yes (single-region)	Rarely	Rarely	No
Proposed system	**Yes**	**Yes (NSGA-II)**	**Yes (Guardian Loop)**	**Yes (parameterised)**	**Yes**

Source: Author's assessment from public documentation; [verify current product features before submission].

The comparison does not claim that existing products lack all of these features in every context; products evolve rapidly and several now offer partial conversational or alert functions. The claim is narrower: none of the surveyed approaches is documented as

combining all five capabilities in a single pipeline specialised for Sri Lankan inter-city travel.

Table 2.3 compares the categories of system discussed above across the capabilities that this thesis requires. The assessment is the author's reading of the literature and of publicly described product features, and it is intended to show where the integration gap lies rather than to rank individual products. Product capabilities change quickly and should be re-checked before submission.

Table 2.3: Capability comparison of categories of travel-planning system

System category	Free-text understanding	Multi-objective package optimisation	Real-time monitoring	Proactive alerts	Sri Lanka terrain modelling
Travel aggregators and map applications	Limited	No	Partial	Partial	No
Generic LLM travel assistants	Yes	No	Limited	No	No
Academic trip design optimisers	No	Yes	No	No	Rarely
Dynamic re-planning research	No	Partial	Yes	Partial	No
Proposed system	Yes	Yes	Yes	Yes	Yes (design)

Source: Author's assessment from the literature review; to be re-verified against current product documentation.

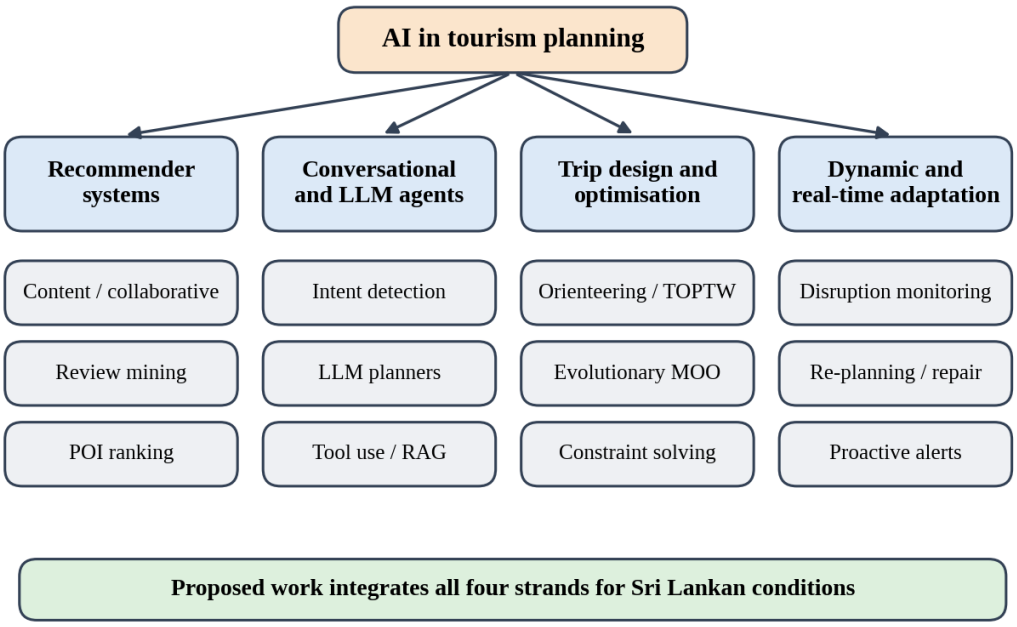


Figure 2.1: Landscape of related work and the position of the proposed system

Figure 2.1 summarises the structure of the review. The four strands are largely developed in isolation, and the proposed system sits at their intersection. Section 2.5 now states the research gap precisely and derives the conceptual framework that guides the design.

2.5 Gap Synthesis and Conceptual Framework

Our review highlights four key takeaways. First, language models are great at understanding open-ended requests, but struggle to reliably handle exact numbers and limits. Second, while optimization algorithms are great at strictly following constraints, they require clean, structured data to work. Third, dynamic optimization tools can quickly fix a travel plan when things change, but they are rarely hooked up to a chat interface for travelers. Finally, regional travel conditions are rarely modeled in detail. To tie all this together, our research framework assigns each of these strengths to a specialized agent and connects them all through a shared plan state, as outlined in Table 2.4.

Table 2.4: Mapping of research gaps to design response and onjectives

Gap	Design response	Objective addressed
Reactive behaviour	Guardian Loop with risk scoring and proactive voice alerts	Objectives 4 and 5
Generation without optimisation	LLM limited to extraction and verbalisation; NSGA-II enforces the budget by constrained domination	Objectives 1 and 2
Weak regional modelling	Terrain-aware travel-time model parameterised by road class, terrain and mode	Objective 3
Fragmented capability	Tri-Agent architecture with shared itinerary state and an integrated evaluation	Objective 6

Source: Author.

The framework rests on the principle of separation of concerns: the language model is used where language flexibility is required, the evolutionary optimiser where constraint satisfaction and trade-off exploration are required, and a transparent scoring rule where explainable triggering is required. Chapter 3 translates this framework into a research methodology.

3 RESEARCH METHODOLOGY

This chapter explains how the research was designed and how it is evaluated. It describes the Design Science Research approach, the data and study materials, the experiments, the metrics, the statistical analysis, and the ethical and reproducibility arrangements.

3.1 Research Design

3.1.1 Design Science Research

The research follows the Design Science Research (DSR) methodology. DSR is appropriate when the principal contribution is a purposeful artefact, in this case a software system that solves an identified problem, and when that artefact is built and then rigorously evaluated [33]. Hevner and colleagues articulate guidelines that include producing a viable artefact, addressing a relevant problem, evaluating the design with rigorous methods, and communicating the result. Peffers and colleagues operationalise these ideas in a six-step process model: problem identification, definition of objectives, design and development, demonstration, evaluation and communication [34]. Figure 3.1 shows how this process is applied here.

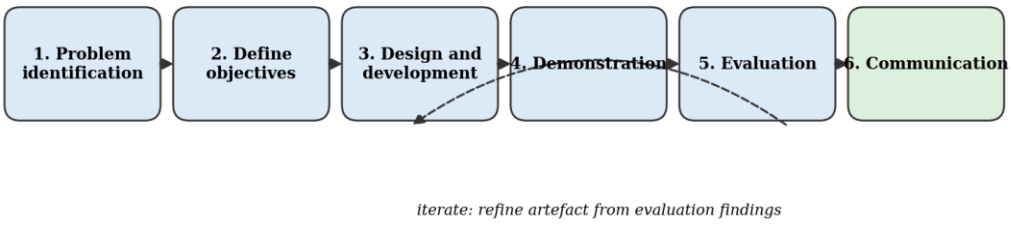


Figure 3.1: Design Science Research process applied in this study, adapted from the process model of Peffers et al.

Table 3.1: Application of the DSR activities in this thesis

DSR activity	Activity in this study	Thesis location
Problem identification	Analysis of planning difficulty for multi-destination trips in Sri Lanka; gap analysis of literature and products	Chapters 1 and 2
Objectives	Six specific objectives; four research questions and hypotheses	Sections 1.5 and 1.6
Design and development	Tri-Agent architecture, NSGA-II formulation, travel-time model, risk scoring, voice alert design; implementation	Chapters 4 and 5
Demonstration	End-to-end worked scenarios, such as a four-day Colombo–Kandy–Nuwara Eliya–Galle trip with an injected rainfall event	Chapters 5 and 6
Evaluation	Controlled experiments, baseline comparison, statistical tests and a usability study	Chapters 3 and 6
Communication	This thesis and the final presentation	All chapters

Source: Author, adapted from [34].

3.1.2 Research strategy

The study uses a mixed approach. Quantitative experiments on simulated scenarios provide controlled and repeatable measurement of extraction accuracy, optimisation quality, feasibility and latency. A small usability study with representative users provides qualitative and quantitative evidence of the usefulness and clarity of the proactive alerts. The combination is appropriate because algorithmic quality can be measured objectively, while the value of a proactive notification is ultimately a human judgement.

3.1.3 Evaluation design principles

- **Hypothesis-driven.** Each experiment tests a hypothesis stated in Section 1.6 with a success criterion fixed in advance.
- **Baseline comparison.** Every claim of improvement is made relative to an explicit baseline evaluated under the same conditions and the same computational budget.
- **Repeatability.** Stochastic algorithms are run multiple times with recorded random seeds, and results are reported with dispersion and confidence intervals rather than single runs.

- **Separation of concerns.** Each agent is first evaluated in isolation, then the integrated pipeline is evaluated end to end, so that the source of any weakness can be located.

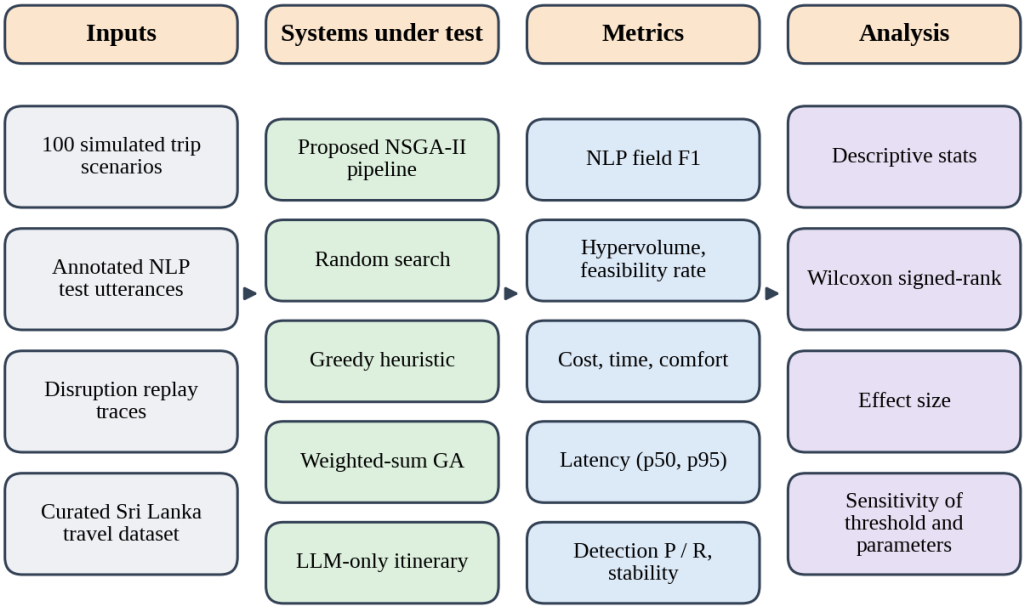


Figure 3.2: Overview of the experimental design: inputs, systems under test, metrics and analysis

Figure 3.2 summarises how the experiments fit together. The same scenario set is given to the proposed pipeline and to each baseline, so that differences in the results can be attributed to the method rather than to the inputs. Where a baseline cannot be run on a scenario (for example because the language model baseline returns an unparseable plan), that outcome is recorded as an infeasible result rather than being discarded, to avoid survivorship bias.

3.2 Data, Participants and Study Materials

3.2.1 Travel knowledge base

The Intelligent Package Generator requires a structured knowledge base of candidate flights, hotels, transport segments and attractions. The base is assembled from publicly available sources and documented snapshots. Table 3.2 lists the entities, principal attributes and intended sources.

Table 3.2: Travel knowledge base entitie and sources

Entity	Principal attributes	Intended source
Attraction	id, name, category, city, latitude, longitude, average visit time, entry fee, rating, indoor/outdoor flag	Public attraction listings and official tourism sources; extends the schema of the proposal dataset
Hotel	id, city, tier, nightly price (LKR), rating, amenities	Public hotel price datasets and dated snapshots
Flight option	origin, destination, departure and arrival time, fare, class	Public fare snapshots for routes such as Dubai–Colombo
Transport segment	from, to, mode (car, train, bus), distance, road class, terrain category, typical duration, fare	Road and rail network data, published timetables and mapping services
Risk reference	historical rainfall, known disruption events, seasonal crowd indices	Historical weather archives and published tourism statistics

Source: Author.

An indicative record structure for attractions is shown in Appendix D. All monetary values are expressed in Sri Lankan rupees (LKR); other currencies are converted using a recorded exchange rate stored with each scenario so that results are reproducible.

3.2.2 Simulated planning scenarios

Representative travel-planning scenarios are used to examine how the proposed system handles different travel requirements. These scenarios reflect common planning situations in Sri Lanka, where travellers may have different budgets, trip durations, destination preferences, transport requirements, and travel styles. Example scenarios may include a short trip to Colombo and Kandy, a multi-destination journey covering Kandy, Nuwara Eliya, and Galle, or a nature-focused trip involving destinations such as Ella.

The scenarios are intended to represent a range of travel-planning needs, from trips with relatively flexible budgets to those with tighter financial limits. They also consider different transport preferences, such as travelling by car, preferring train journeys, or allowing the system to consider alternative transport options. These variations provide a practical basis for examining how the chatbot interprets user requirements and how the route-ranking module compares available travel options. The selected scenarios can be used to observe whether the system identifies the main travel constraints and produces route rankings that reflect its defined objectives.

Where sufficient candidate data are available, scenarios with restrictive budgets can also be used to examine whether the system identifies suitable alternatives or indicates that no feasible option is available. The evaluation is limited to the scenarios and system outputs actually examined during development; therefore, no claim is made that a statistically representative set of 100 scenarios was generated or that formal stratified sampling was performed.

Table 3.3: Scenario factors and levels

Factor	Levels
Budget (LKR)	150,000; 250,000; 400,000; 600,000
Duration (days)	3; 4; 5; 7
Destination set	{Colombo, Kandy}; {Kandy, Nuwara Eliya, Galle}; {Colombo, Kandy, Ella, Galle}; {Sigiriya, Kandy, Nuwara Eliya, Mirissa}
Transport preference	Transport Service

Source: Author.

3.2.3 Annotated utterance set for the NLP Analyzer

The NLP Analyzer is designed to understand travel requests expressed in natural language and identify the information required for trip planning. Travellers may describe their requirements in different ways, using formal sentences, short phrases, informal expressions, or messages that contain several requirements at once. For example, a user may specify a destination, travel duration, and preferred transport method within a single message.

During the development of the prototype, sample travel requests are used to examine how the chatbot interprets these requirements. The examples cover common information such as the starting location, destination, duration of the trip, budget, and transport preference. They also help illustrate how the system handles variations in wording and requests in which some details are not explicitly provided.

The extracted information is used to support the subsequent travel-planning and route-ranking processes. These examples provide a practical way to inspect the behaviour of the NLP Analyzer during development. However, a formal annotated corpus with a separate test set has not been established as part of the completed evaluation. Therefore, the study does not report a corpus size, inter-annotator

agreement, or quantitative NLP accuracy unless these measures are obtained through a separate evaluation.

3.2.4 Disruption scenarios for the Adaptive Engine

The Adaptive Engine is intended to support travel planning when conditions may affect the original itinerary. Factors such as adverse weather, traffic congestion, transport delays, road closures, and increased visitor numbers can influence the suitability of a planned route. Considering these factors is important when providing travel recommendations that remain relevant to changing conditions.

For the proposed system, these disruption types provide a basis for describing how travel plans could be assessed and updated when new information becomes available. For example, heavy rainfall may affect the suitability of an outdoor attraction, while a road closure or transport delay may require a traveller to consider an alternative route or adjust the planned schedule.

The evaluation of this component must be based on the disruption-handling features implemented in the prototype. Historical signal replay and the systematic injection of disruption events are not claimed as completed experiments unless supported by implementation and test records. Accordingly, this section describes the intended role of disruption information in adaptive travel planning rather than presenting unverified results about detection accuracy, response time, or false-alarm rates.

3.2.5 Participants for the usability study

The intended users of the proposed system are travellers who need assistance with planning trips in Sri Lanka. These users may have different destination preferences, budgets, travel durations, and transport requirements. The conversational interface is designed to make travel planning more accessible by allowing users to express their requirements in natural language.

The completed work focuses on the development of the prototype and the examination of its technical behaviour using available sample inputs. A formal usability study involving recruited human participants has not been completed as part

of this evaluation. Therefore, no participant count, demographic profile, recruitment method, questionnaire score, or user-satisfaction result is reported.
A future usability study could involve local travellers, international students, or tourism-industry representatives completing representative travel-planning tasks and providing feedback on the clarity of the chatbot, the relevance of its recommendations, and the ease of interaction. Such an evaluation would provide additional evidence about the practical usefulness of the system. However, these activities are treated as future evaluation work rather than completed research procedures.

3.3 Experimental and Study Procedure

Seven experiments, listed in Table 3.4, are carried out. Experiments E1 to E3 evaluate the agents separately; E4 to E6 evaluate planning and adaptation; E7 evaluates the integrated user experience.

Table 3.4: Experiments, hypotheses and baselines

ID	Purpose	Hypothesis	Baselines or comparators
E1	Constraint extraction accuracy	H1	Rule-based regular-expression extractor; zero-shot versus few-shot prompting; GPT-4o versus Llama 3 variants
E2	Travel-time model accuracy	H3	Constant-speed model (distance divided by one average speed)
E3	Optimisation quality	H2	Random search; greedy cheapest-first construction; weighted-sum single-objective GA; LLM-only itinerary
E4	Budget feasibility and efficiency	H2	Same as E3
E5	Risk detection and re-optimisation	H4	Fixed-rule alerting (single threshold on rainfall); no-adaptation control
E6	Response time and scalability	H4	Cold-start versus warm-start re-optimisation; requirement of 5 s
E7	Usability and perceived usefulness	–	Descriptive; System Usability Scale

Source: Author.

3.3.1 E1: constraint extraction

Each utterance in the held-out test set is passed to the Analyzer, and the JSON output is compared with the annotation slot by slot. Prompts are fixed using the development

subset only. The comparison between prompting strategies and model variants uses the same test set and the same decoding settings (temperature fixed at a low value) so that differences reflect the method and not randomness.

3.3.2 E2: travel-time model

For each of a set of origin–destination pairs and modes, the terrain-aware estimate and the constant-speed estimate are compared with reference journey times obtained from a routing service and, for trains, from the published timetable. Parameters of the terrain-aware model are fitted on a calibration subset of the pairs and evaluated on the remainder to avoid overfitting.

3.3.3 E3 and E4: optimisation quality and feasibility

For every scenario, each stochastic algorithm is run 30 times with different seeds and an identical budget of objective-function evaluations. Deterministic methods are run once. The primary output is the non-dominated set returned by each run. The initial NSGA-II settings are shown in Table 3.5; they are tuned on a small pilot subset of scenarios that is excluded from the reported experiments.

Table 3.5: Initial NSGA-II settings

Parameter	Initial setting
Population size N	100
Maximum generations G_{max}	100 (early stop if hypervolume improves by less than 0.1% over 15 generations)
Crossover	Order crossover (OX) for visiting order; uniform crossover for integer genes; probability 0.9
Mutation	Swap mutation for order; random-reset mutation for integer genes; per-gene probability 1/n
Selection	Binary tournament with crowded comparison
Constraint handling	Constrained-domination principle
Initialisation	Random with a fraction seeded from greedy solutions
Runs per scenario	30 independent runs

Source: Author.

3.3.4 E5: risk detection and re-optimisation

Each disruption scenario is replayed against the Guardian Loop. For each signal batch the engine computes the segment risk, applies the trigger rule, and, if triggered, re-optimises. The output is compared with the ground-truth label for each affected

segment. Detection delay is the time between the true start of a disruption in the stream and the first alert. The quality of the repaired plan is measured by feasibility, additional cost and the number of itinerary changes.

3.3.5 E6: response time

Wall-clock time is recorded for each stage (LLM extraction, optimisation, re-optimisation and message generation) on a specified hardware and deployment configuration. Median, 95th-percentile and maximum values are reported. Because a language model call may dominate the latency, the end-to-end figure is reported both with and without the call.

3.3.6 E7: usability study

Each participant receives a brief introduction, completes a planning task through free-text input, and then experiences a simulated disruption with a voice alert. Afterwards they complete the System Usability Scale (SUS) questionnaire [35] and answer questions about the clarity, timeliness and usefulness of the alert. Observations and comments are recorded anonymously.

3.4 Measures and Evaluation Metrics

3.4.1 Extraction metrics

For slot-level evaluation, a predicted slot value is a true positive if it equals the annotated value after normalisation, a false positive if it is predicted but wrong or unsupported, and a false negative if an annotated value is missing. Precision, recall and F1 are:

$$P = TP / (TP + FP), \quad R = TP / (TP + FN), \quad F1 = 2PR / (P + R) \tag{3.1}$$

Results are reported per slot (budget, duration, destinations, transport, style) and micro-averaged. Exact-match accuracy per utterance, and the rate of unsupported (hallucinated) slot values, are also reported.

3.4.2 Travel-time error

$$MAE = (1/n)\ \Sigma\ |\ \hat{t}_i - t_i\ |, \quad MAPE = (100/n)\ \Sigma\ |\ \hat{t}_i - t_i\ |\ /\ t_i \tag{3.2}$$

where $\hat{t}_i$ is the model estimate and t_i the reference time for pair i.

3.4.3 Optimisation metrics

- **Hypervolume (HV).** Computed after normalising objectives using fixed ideal and nadir points per scenario and a common reference point, so that values are comparable across algorithms [13].
- **Constraint-violation rate.** The proportion of returned solutions with total cost above the budget.
- **Budget utilisation.** The ratio of spent budget to available budget for the recommended solution, interpreted together with comfort and time, since spending the full budget is not by itself desirable.
- **Convergence rate.** The number of generations, or function evaluations, needed to reach 95% of the final hypervolume.
- **Feasibility rate.** The proportion of scenarios for which at least one feasible package is returned, compared with the proportion for which a feasible package is known to exist.

3.4.4 Risk-detection metrics

Precision, recall and F1 are computed over segment-level alert decisions. Additional measures are detection delay, false-alarm rate on control scenarios, and plan stability, defined as the fraction of unaffected, not-yet-completed segments that remain unchanged after re-optimisation.

3.4.5 Performance and usability metrics

System response time is reported per stage and end to end. Usability is summarised by the SUS score (0 to 100), by task completion, and by Likert-scale ratings of alert clarity and timeliness.

3.4.6 Sensitivity of the risk threshold and algorithm parameters

The risk threshold and the evolutionary parameters are design choices whose values were set from reasoning rather than from data. To test whether conclusions depend on them, each is varied over a stated range while the others are held fixed. The threshold θ is varied over 0.5, 0.6, 0.7, 0.8 and 0.9 and the persistence k over 1, 2 and 3 polls; the trade-off between detection delay and false alerts is then plotted as a precision–

recall curve. For the optimiser, the population size N is varied over 50, 100 and 200, the number of generations over 50, 100 and 200, and the crossover and mutation probabilities over three levels each. Because a full factorial design would be expensive, a fractional design on a random subset of 20 scenarios is used, and only the parameter settings that matter are carried to the full scenario set.

A result is reported as robust only if the ranking of methods is unchanged across the parameter ranges tested. If the ranking changes, the thesis reports the conditions under which it changes. This practice protects against the common weakness of evolutionary computation studies in which a proposed method is compared against baselines whose parameters have not been tuned with equal effort. To reduce that risk, the weighted-sum GA and random-search baselines are given the same evaluation budget as NSGA-II.

3.5 Data Analysis

Algorithm comparisons are paired by scenario. For each scenario and algorithm, the median hypervolume over the 30 runs is used as the summary statistic. The Wilcoxon signed-rank test [36] is applied to compare NSGA-II with each baseline, with the Holm correction for multiple comparisons, following the recommended practice for comparing evolutionary algorithms [37]. Effect size is reported using the Vargha–Delaney A_{12} statistic [38], which gives the probability that a random run of one algorithm outperforms a random run of another; values above 0.71 are conventionally considered large. Ninety-five per cent confidence intervals for proportions (such as recall) use the Wilson interval, and for means use the bootstrap. The significance level is $\alpha = 0.05$ before correction.

For the usability study, which has a small sample, results are reported descriptively with confidence intervals, and no inferential claims are made about the population of tourists.

3.5.1 Experiment schedule and dependencies

The experiments depend on each other and must be run in a fixed order. Table 3.6 shows the order and the artefact each experiment requires from the previous one.

Placing the schedule in the methodology makes the dependency explicit and gives the reader a way to judge whether the work packages in the project plan are realistic.

Table 3.6: Order and dependencies of the experiments

Order	Experiment	Requires	Produces
1	E2 travel-time model	Routing reference data	Calibrated parameters (Table 4.6)
2	E1 constraint extraction	Annotated utterance set	Chosen model and prompt
3	E6 response time (preliminary)	Working pipeline	Latency budget check
4	E3 and E4 optimisation	Calibrated model; scenario set	Hypervolume, feasibility results
5	E5 risk detection	Disruption scenarios; chosen threshold	Detection and stability results
6	E7 usability	Stable prototype; consent forms	SUS scores and themes

Source: Author.

3.6 Ethics, Privacy, Security and Responsible AI

3.6.1 Data protection

The system is designed to comply with the principles of the Personal Data Protection Act, No. 9 of 2022 of Sri Lanka [39]: lawful and transparent processing, purpose limitation, data minimisation and security. Experimental scenarios are synthetic and contain no personal data. For the usability study, participants give informed consent, no names or contact details are recorded with responses, and collected data are stored in access-controlled storage and reported only in aggregate.

3.6.2 Responsible use of language models

- **Grounded outputs.** The language model never invents prices, schedules or availability; it only extracts what the user says and verbalises facts supplied by the optimiser and the monitoring feeds.
- **Validation and confirmation.** Extracted values are validated against a schema and ranges, and uncertain or ambiguous values are confirmed with the user.
- **Transparency.** The interface identifies the assistant as an AI system and indicates when a plan has been changed automatically and why.

- **Safety.** Alerts concerning weather or hazards are advisory and cite their source and time; the system does not replace official warnings from authorities.
- **Data sent to third-party model providers** is limited to the minimum text needed for extraction and excludes identifiers.

3.6.3 Security

External APIs are accessed over encrypted channels with keys stored in server-side secrets rather than in the mobile client. Inputs are validated before reaching the optimiser, and model outputs are parsed as data and never executed. These measures are described further in Section 4.4.

3.7 Reproducibility and Research Artefacts

To support reproducibility, the following artefacts are preserved:

- Source code.
- API configuration structure.
- Environment configuration templates.
- Optimization logic.
- Test cases.
- Sample input and output data.
- Evaluation results.
- Architecture diagrams.
- Documentation.

API keys and private credentials should not be included in publicly shared source code.

4 PRODUCT AND SOLUTION DESIGN

This chapter specifies what the system must do and how it is designed. It presents the requirements, the architecture of the component within the complete group solution, the main design alternatives and the reasons for the decisions taken, and the detailed design of each agent.

4.1 Requirements

4.1.1 Stakeholders

The principal stakeholders are international tourists planning a trip to Sri Lanka, local tourists planning domestic trips, travel agencies and tour operators that wish to produce packages efficiently, and tourism authorities that have an interest in visitor safety and distribution. A secondary group is made up of the suppliers (airlines, transport operators) whose offerings are represented in the knowledge base. Requirements were derived from the problem analysis in Chapter 1, the gaps identified in Chapter 2, and the scenarios in the original project proposal.

4.1.2 Functional requirements

Table 4.1: Functional requirements

ID	Requirement	Priority	Verification
FR1	The system shall extract budget, duration, destinations, transport preference and travel style from a free-text request.	Must	E1; unit tests
FR2	The system shall normalise Sri Lankan and international monetary expressions (for example "4 lakhs", "400k LKR", "USD 1,200") to a single currency.	Must	Unit tests; E1
FR3	The system shall request clarification when a required field is missing or ambiguous rather than assuming a value.	Must	Scenario tests
FR4	The system shall generate a complete package comprising flight, accommodation, inter-city transport and daily activities.	Must	E3; integration tests
FR5	The system shall guarantee that the total package cost does not exceed the stated budget, or report that no feasible package exists.	Must	E3, E4
FR6	The system shall estimate inter-city travel times using road class, terrain and mode.	Must	E2
FR7	The system shall present a small set of Pareto-efficient alternatives with the cost, time and comfort of each.	Should	E3; usability

ID	Requirement	Priority	Verification
FR8	The system shall monitor weather, flight status, rail status, traffic and crowd level for the remaining segments of the itinerary.	Must	E5; integration tests
FR9	The system shall compute a risk score for each remaining segment and trigger re-optimisation when the score meets the trigger rule.	Must	E5
FR10	The system shall preserve completed segments and minimise unnecessary changes when re-optimising.	Should	E5 (plan stability)
FR11	The system shall notify the traveller proactively by voice and push notification, explaining what changed and why.	Must	E5, E7
FR12	The system shall allow the traveller to accept or reject a proposed change and shall record the decision.	Should	Usability tests

Source: Author, refined from the six requirements in the project proposal.

4.1.3 Non-functional requirements

Table 4.2: Non-functional requirements and measurable targets

ID	Quality attribute	Requirement (target)	Verification
NFR1	Performance	Package generation, including extraction, completes in under 5 s at the 95th percentile on the reference deployment.	E6
NFR2	Performance	Warm-start re-optimisation completes in under 5 s at the median.	E6
NFR3	Correctness	Zero budget violations among recommended packages.	E3, E4
NFR4	Reliability	If an external feed is unavailable, the engine continues with the last valid value for up to a defined staleness limit and flags the signal as stale; the failure of one feed does not stop the loop.	Fault-injection tests
NFR5	Security	All external and client–server traffic uses HTTPS; API keys are held server-side; inputs are validated.	Security review
NFR6	Privacy	Personal data is minimised and processed in accordance with the PDPA No. 9 of 2022.	Design review
NFR7	Scalability	Agents are stateless services that can be scaled independently; shared state is held in a database.	Architecture review; load test if time permits
NFR8	Usability	Mean SUS score reaches an acceptable level (target of at least 68, the commonly cited average).	E7
NFR9	Explainability	Every automatic plan change states its cause and data source.	Inspection; E7
NFR10	Maintainability	Parameters (weights, thresholds, speeds) are stored in configuration, not code.	Code review

Source: Author.

The 5 s requirement in NFR1 is demanding because a language-model call and an evolutionary run both consume time. To make it testable, the design allocates a latency budget to each stage, as shown in Table 4.3. These allocations are design targets that Experiment E6 will confirm or refute; they are not measurements.

Table 4.3: Latency budget allocated to each stage (design targets)

Stage	Budget (s)	Main lever
Network and API gateway	0.5	Regional deployment
LLM extraction and validation	2.0	Short prompt; low-latency model; streaming
Knowledge-base retrieval and pre-computation of leg times	0.5	Pre-computed matrices; caching
NSGA-II optimisation	1.5	Vectorised evaluation; early stopping
Response composition	0.5	Template-based formatting
Total	**5.0**	

Source: Author.

4.2 Component Architecture within the Complete Solution

4.2.1 Overview

The component is implemented as a Tri-Agent architecture, illustrated in Figure 4.1. The traveller interacts through a mobile conversational interface that supports text and voice. The three agents communicate through a shared itinerary and state store rather than through direct calls, which decouples them and allows each to be scaled and tested independently.

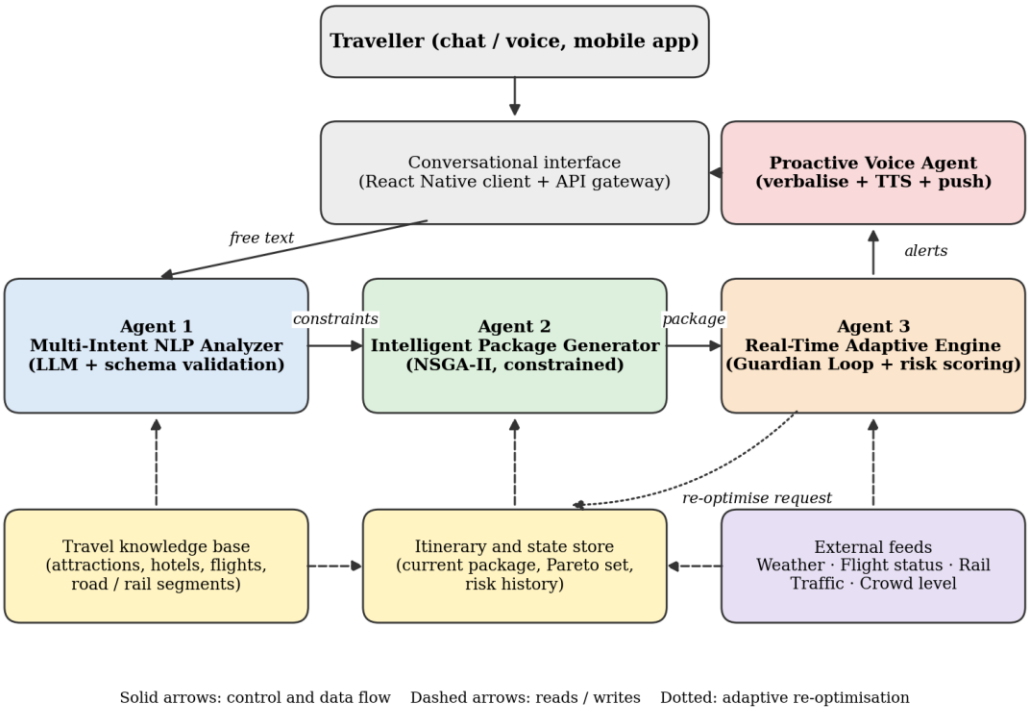


Figure 4.1: Architecture of the Tri-Agent system and its data stores

4.2.2 Responsibilities of the agents

Table 4.4: Responsibilities and interfaces of the three agents

Agent	Input	Output
Multi-Intent NLP Analyzer	User utterance (text, or transcribed speech); conversation context	Validated constraint object (JSON); clarification questions; detected intents
Intelligent Package Generator	Constraint object; knowledge base; terrain-aware time model	Pareto set of packages; recommended package; stored itinerary
Real-Time Adaptive Engine	Stored itinerary; live feeds; configuration of weights and thresholds	Risk scores; re-optimised alternatives; structured change events for the Voice Agent
Proactive Voice Agent	Structured change event	Spoken and written alert; user decision

Source: Author.

4.2.3 Two operating phases

The system operates in two phases. In **Phase 1, personalised package generation**, the analyzer converts the request into constraints, the generator solves the optimisation problem, and the recommended package is stored and shown to the traveller. In **Phase 2, continuous monitoring and adaptive re-optimisation**, the Guardian Loop runs in parallel from the moment the package is accepted. If the trigger rule fires, the engine

freezes completed segments, re-optimises the remainder, and the Voice Agent informs the traveller.

4.2.4 Integration with the rest of the group solution

The AI Conversational Travel Analyzer is built to work hand-in-hand with the rest of the this framework, tapping into live travel intelligence to sharpen both package creation and dynamic recommendations.

By connecting smoothly with specialized modules—such as the emotion recognition component (IT22216014), the driver and vehicle safety component (IT22176974), and the crowd management component (IT22607614)—the system unlocks powerful new ways to enrich trip planning:

- Traveler Mood Insights: Pulling in emotional state feedback to tailor itineraries to the user's current energy and mood.
- Safety & Security Alerts: Integrating real-time driving and vehicle safety warnings to keep routes secure.
- Crowd Density Tracking: Factoring in crowd data to steer travelers away from congested hotspots.

These modules talk to each other seamlessly through well-defined software interfaces—relying on standard REST API endpoints or clean, structured data models to keep the entire ecosystem synchronized.

4.2.5 Deployment view

Each agent is packaged as an independent service behind an API gateway, in line with a cloud microservice model [40]. The mobile client is built with React Native so that a single code base serves Android and iOS. Background monitoring runs as a scheduled or event-driven worker on the server, so that it continues even when the application is closed, and alerts are delivered as push notifications that open the voice interaction.

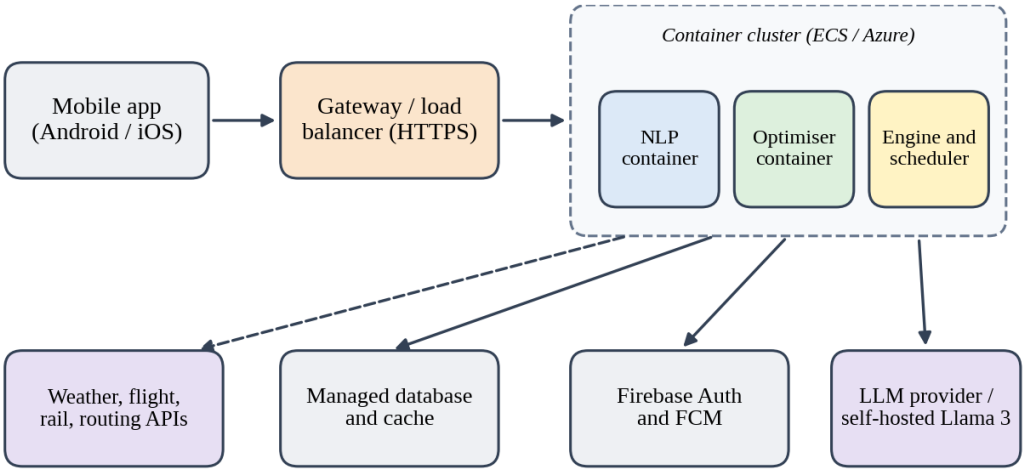


Figure 4.2: Deployment view of the system on a cloud container platform

Figure 4.2 shows the intended deployment. The three services are packaged as separate containers behind a gateway, which allows the optimiser, which is processor-bound, to be scaled independently of the language-model service, which is latency-bound. State is held in managed storage rather than in the containers, so that any container can be replaced without losing a traveller's itinerary. This follows the usual microservice guidance on keeping services small, independently deployable and loosely coupled [41], and it matches the cloud service model described by the National Institute of Standards and Technology [40].

4.3 Design Alternatives and Decisions

Several design decisions were taken after considering alternatives. Table 4.5 summarises the principal ones.

Table 4.5: Design alternatives and decisions

Decision	Alternatives considered	Reasoning	Choice
Optimisation method	Mixed-integer programming; weighted-sum GA; NSGA-II; NSGA-III; MOEA/D; LLM-only	The objectives are conflicting and some (travel time with terrain factors, comfort) are non-linear; the traveller benefits from a set of trade-offs rather than a single weighted answer; there are three objectives, within the range for which NSGA-II is well suited; a hard budget constraint is handled directly by constrained domination. LLM-only planning cannot guarantee constraints [5].	NSGA-II
Constraint handling	Penalty function; repair only; constrained domination	Penalty weights are hard to tune and can distort the front; constrained domination needs no extra parameter [4]; light repair is kept only for structural validity (nights sum).	Constrained domination plus structural repair
Language model	Fine-tuned BERT slot filler; GPT-4o; Llama 3 (open weights)	Few labelled travel data are available, and the slot schema may change; prompting a language model avoids task-specific training [19]. Both proprietary and open-weight models are evaluated, because open weights allow on-premise deployment and cost control [21], [22].	Schema-constrained LLM; model compared in E1
Role of the LLM	End-to-end planner; extractor and verbaliser only	End-to-end planning is unreliable for constraints; confining the model to language tasks makes numerical behaviour verifiable and reduces hallucination [23].	Extractor and verbaliser
Disruption trigger	Machine-learned predictor; transparent weighted risk score	No labelled disruption data exist for this domain; a transparent score is explainable, tunable, and testable by sensitivity analysis.	Weighted risk score with hard-event override
Re-planning strategy	Re-plan from scratch; warm-start repair	Warm start re-uses the previous solution to adapt faster and keep the plan stable [28], [29].	Warm-start with plan-change penalty
Mobile client	Native Android and iOS; Flutter; React Native	Single code base, wide library support, and a notification and speech ecosystem.	React Native
Backend language	Python; Node.js; Java	Python offers mature evolutionary-computation and NLP libraries [14], [15].	Python

Source: Author.

4.4 Detailed Design

4.4.1 Multi-Intent NLP Analyzer

Intents. The analyzer recognises a small, closed set of intents: PLAN_TRIP (create a package), MODIFY_PLAN (change an existing constraint or item), ASK_STATUS (ask about weather, delays or the current plan), EXPLAIN (ask why a change was made) and ACCEPT_REJECT (respond to a proposed change). A single utterance may contain several intents, for example "Plan four days in Kandy and Galle for four lakhs, and tell me if it will rain in Galle."

Slots. The target output is a JSON object with the fields in the schema below. Fields that the user did not state are returned as null, and the model lists them in missing_fields. The model is instructed to copy the supporting text span for each value, which allows a post-hoc check that the value is grounded in the input.

```
{
  "intents": ["PLAN_TRIP"],
  "budget": {"amount": 400000, "currency": "LKR", "span": "4 lakhs"},
  "duration_days": 4,
  "origin": "Dubai",
  "destinations": ["Kandy", "Nuwara Eliya", "Galle"],
  "must_visit": [],
  "missing_fields": [],
  "ambiguous_fields": []
}
```

Pipeline. (1) A prompt containing the schema, short definitions and a few worked examples is sent to the model with a low decoding temperature. (2) The reply is parsed as JSON and validated against the schema; failures trigger one automatic retry with the validation error appended. (3) A normaliser converts monetary expressions to LKR (the unit "lakh" equals 100,000), resolves duration abbreviations such as "10d", maps place names to knowledge-base identifiers using fuzzy matching, and applies range checks (for example a duration of at least one day and a positive budget). (4) A grounding check verifies that each extracted value has a supporting span in the input; values without support are discarded and added to missing_fields. (5) If required fields are missing or ambiguous, the dialogue manager asks a single, specific clarification question; otherwise the constraint object is passed to the generator.

Design rationale. Placing deterministic validation and normalisation around the language model addresses the principal risk of LLM extraction, namely unsupported or inconsistent values [23]. Because arithmetic such as unit conversion is performed in code rather than by the model, the numerical result is verifiable.

4.4.2 Intelligent Package Generator

4.4.2.1 Problem formulation

The Traveller Constraint Satisfaction Problem is formulated as follows. Given a budget B, a number of days D, an ordered or unordered set of required destinations V, transport preferences and a style profile, choose a package x consisting of a flight, a visiting order of the destinations, the number of nights in each destination, a hotel tier for each stay, a transport mode for each leg, and an activity set for each day, such that three objectives are optimised and the constraints are met.

4.4.2.2 Encoding

A candidate package is encoded as a mixed chromosome of six genes, shown in Figure 4.3. The visiting-order gene is a permutation of the destination set; the others are integer vectors or bit masks. Structural repair ensures that nights sum to D − 1 and that every required destination is visited at least once.

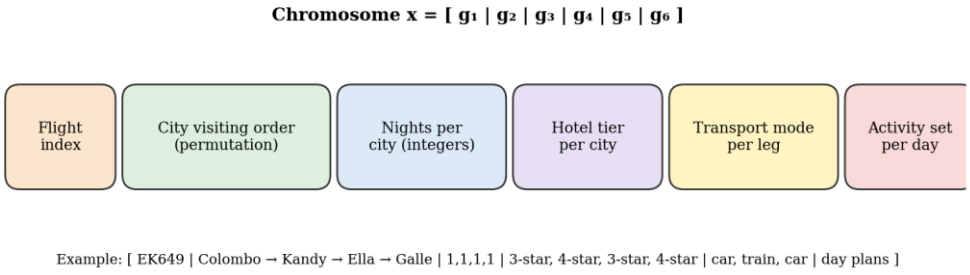


Figure 4.3: Chromosome encoding of a travel package

4.4.2.3 Objectives

Three objectives are defined, all expressed for minimisation.

$$f_1(x) = C_flight + \Sigma_i\, n_i \cdot h(c_i, \tau_i) + \Sigma_j\, p(m_j, d_j) + \Sigma_k\, e(a_k) + D \cdot m_0 \qquad (4.1)$$

where C_flight is the fare, n_i the nights in the i-th city, $h(c_i, \tau_i)$ the nightly price of tier τ_i in city c_i, $p(m_j, d_j)$ the fare for leg j of distance d_j by mode m_j, $e(a_k)$ the entry fee of activity a_k, and m_0 a daily allowance for meals and incidentals. Total travel time is:

$$f_2(x) = \Sigma_j \, T(m_j, r_j, \tau_j, d_j) + t_transfer \tag{4.2}$$

where T is the terrain-aware travel-time function defined below, for leg j with road class r_j and terrain category τ_j, and t_transfer accounts for airport transfers. The third objective captures comfort and preference fit:

$$f_3(x) = -\left[\, \alpha \cdot C_hotel(x) + \beta \cdot C_transport(x) + \gamma \cdot S_pref(x) \,\right] \tag{4.3}$$

where each term is normalised to [0, 1], C_hotel rewards higher-quality stays, C_transport rewards comfortable modes and penalises long uninterrupted journeys, and S_pref measures the match between selected activities and the traveller's style and interests (for example weighting ratings by category match). The weights α, β and γ sum to one and are fixed by configuration; they affect the shape of f_3 but, unlike a weighted-sum approach, they do not combine cost and time with it.

4.4.2.4 Constraints

$$g_1(x) = f_1(x) - B \leq 0 \tag{4.4}$$

- **Budget:** total cost does not exceed the budget (g_1).
- **Daily load:** on each day, travel time plus activity time does not exceed a maximum H_max (initially 12 hours) (g_2).
- **Coverage:** every must-visit destination is included (g_3).
- **Schedule feasibility:** the first hotel check-in is consistent with the arrival time of the chosen flight, and the departure leg allows the return flight (g_4).

The overall constraint violation is $CV(x) = \Sigma_i \max(0, g_i(x))$ with each term normalised by a scale value. NSGA-II uses CV in the constrained-domination rule [4].

4.4.2.5 Terrain-aware travel-time model

For a road leg consisting of sections s with distance d_s, road class r_s and terrain category τ_s, the expected travel time is

$$T_road = \Sigma_s \, d_s \,/\, \left(v(r_s) \cdot \varphi(\tau_s) \right) + t_stop \tag{4.5}$$

where v(r) is the base speed for road class r, $\varphi(\tau) \in (0, 1]$ a terrain factor that reduces speed for rolling, hilly and mountainous sections, and t_stop an allowance for rest stops. For rail legs, the timetable duration is used and increased by an expected delay allowance. The initial parameter values are in Table 4.6. They are explicitly **assumed starting values**, not established facts; Experiment E2 fits them to reference journey times, and the fitted values and their error are reported in Chapter 6.

Table 4.6: Initial parameters of the terrain-aware travel-time model

Parameter	Initial value	Note
Base speed v, expressway	[to be calibrated, e.g., 70–80 km/h]	Average, including exits
Base speed v, A-class road	[to be calibrated, e.g., 40–50 km/h]	Average through towns
Base speed v, B-class road	[to be calibrated, e.g., 30–35 km/h]	Rural
Terrain factor φ, flat / rolling / hilly / mountainous	[to be calibrated, e.g., 1.0 / 0.85 / 0.7 / 0.55]	Applied to base speed
Rest-stop allowance t_stop	[to be calibrated, e.g., 15 min per 2 h]	Per leg
Rail expected-delay allowance	[to be calibrated from timetable and delay data]	Per rail leg

Source: Author. Values in brackets are placeholders to be replaced by calibrated values.

4.4.2.6 Evolutionary operators and algorithm

Order crossover (OX) and swap mutation are applied to the visiting-order gene, uniform crossover and random-reset mutation to integer genes, and bit-flip mutation to activity masks. Parents are chosen by binary tournament with crowded comparison. The overall procedure, in which the elements of NSGA-II are adapted to the mixed encoding, is shown in Figure 4.4 and in Algorithm 1.

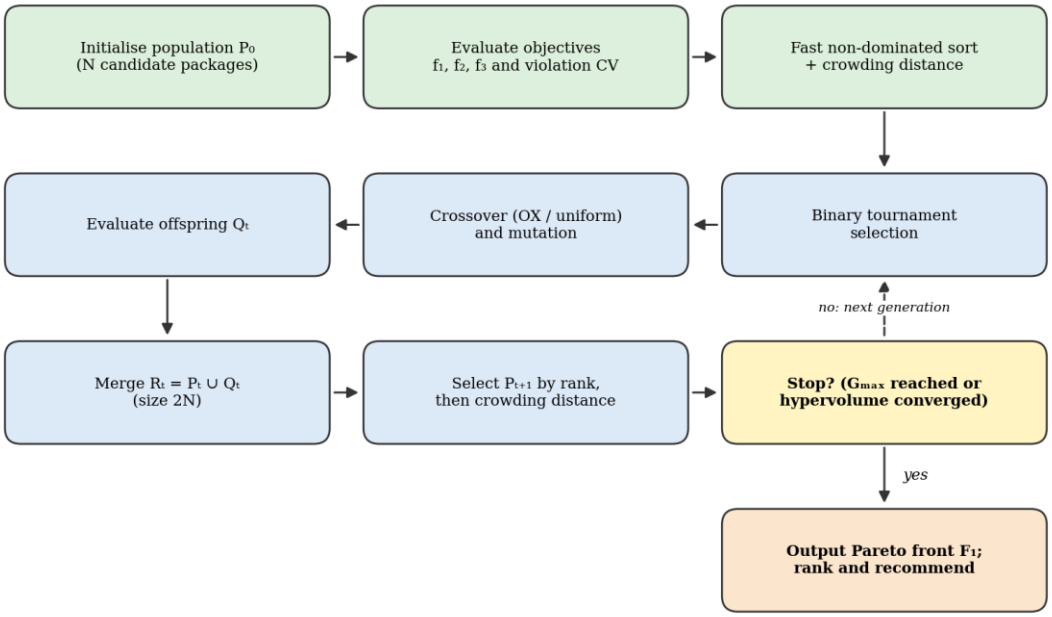


Figure 4.4: NSGA-II procedure used by the Intelligent Package Generator

```
Algorithm 1: Constrained NSGA-II for package generation
Input: constraints C, knowledge base K, N, Gmax, pc, pm
 1  P0 <- InitPopulation(C, K, N)            // random + greedy seeds
 2  Evaluate(P0)                              // f1, f2, f3, CV
 3  for t = 0 .. Gmax-1:
 4      Q <- {}
 5      while |Q| < N:
 6          p1, p2 <- TournamentSelect(P_t)   // rank, then crowding
 7          c1, c2 <- Crossover(p1, p2, pc)
 8          Mutate(c1, pm); Mutate(c2, pm)
 9          Repair(c1); Repair(c2)            // nights sum, coverage
10          Q <- Q U {c1, c2}
11      Evaluate(Q)
12      R <- P_t U Q
13      F <- ConstrainedNonDominatedSort(R)   // feasible first, then
CV
14      P_{t+1} <- FillByFront(F, N)          // crowding distance on
last front
15      if HVImprovement(P_{t+1}) < eps for 15 generations: break
16  return F1                                 // Pareto front of
feasible solutions
```

4.4.2.7 Choosing the recommended package

The Pareto front can contain many solutions. To present a small number of options, the generator (a) discards dominated and infeasible solutions, (b) identifies a knee solution, the point of the front with the largest distance from the line joining its extreme points in normalised objective space, and (c) selects two further solutions that favour the cheapest and the most comfortable end of the front. The knee solution is

highlighted as the default recommendation, and the traveller can inspect the others. If no feasible solution exists, the system reports the infeasibility and proposes the smallest relaxation, for example the smallest budget increase or the removal of the least important destination.

4.4.2.8 Worked example of Pareto selection

To make the selection procedure concrete, consider a four-day trip with a budget of LKR 400,000 for which the generator has produced the seven candidate packages in Table 4.7. The figures are **illustrative values chosen to explain the procedure; they are not experimental results**. Package F has the lowest travel time and highest comfort but costs LKR 421,000, which exceeds the budget by LKR 21,000, so under constrained domination it is infeasible and cannot be recommended. Among the six feasible packages, package E is dominated by package A, which is cheaper, faster and more comfortable. This leaves five non-dominated packages A, B, C, D and G.

Table 4.7: Illustrative candidate packages for the worked example (not experimental data)

Package	Cost (LKR)	Travel time (h)	Comfort score	Status
A	372,000	15.0	0.74	Non-dominated; knee
B	318,000	16.5	0.69	Non-dominated
C	398,000	14.5	0.86	Non-dominated; most comfortable
D	241,000	19.5	0.52	Non-dominated; cheapest
E	385,000	15.5	0.73	Dominated by A
F	421,000	13.5	0.90	Infeasible (over budget by 21,000)
G	345,000	13.8	0.60	Non-dominated; fastest

Source: Illustrative example constructed by the author.

The three extreme packages are D (cheapest), G (fastest) and C (most comfortable). After min–max normalisation of the three objectives over the non-dominated set, these three points define a plane in objective space. The knee is the point lying furthest on the favourable side of that plane. In this example package A is the only point that lies on the favourable side, at a normalised distance of about 0.05, while package B lies on the unfavourable side. The system therefore highlights package A, which costs LKR 372,000, takes 15 hours in transit and has comfort 0.74, as the default recommendation, and offers C, D and G as the comfort-first, cost-first and time-first alternatives. This

presentation matches requirement FR7: the traveller sees a small number of well-separated trade-offs instead of a long list.

The example also shows why dominated-solution filtering matters. Without it, package E might be shown alongside package A even though no traveller could prefer it. Conversely, the example shows why constraint handling must be explicit: package F is the most attractive on two objectives, and a weighted-sum method with a soft budget penalty could return it if the penalty weight were set too low. Constrained domination removes this failure mode by construction.

4.4.2.9 Feasibility of the reference trip

The reference request in the proposal, a four-day trip covering Kandy, Nuwara Eliya and Galle within LKR 400,000, deserves a feasibility discussion because it is representative of the requests the system must handle. The three destinations lie in different parts of the island, and moving between them involves some of the slowest road and rail segments in the country. Even before costs are considered, the available days limit how many nights can be allocated and how long can be spent in transit. The daily travel-time constraint of the model (Section 4.4.2.4) will therefore prune many combinations. It is quite possible that the best response to this request is not a perfect package but a set of trade-offs, together with an explanation that all three destinations cannot be visited comfortably in four days. The system is designed to say so, to propose the smallest relaxation, and to let the traveller choose. This behaviour, which a purely generative assistant is unlikely to show reliably [5], is a strength of the constraint-based formulation.

4.4.3 Real-Time Adaptive Engine (Guardian Loop)

4.4.3.1 Signals and risk factors

The engine consumes five kinds of signal. Each is converted to a risk factor r_k in the range [0, 1] by a monotonic function, as in Table 4.8. The functions are deliberately simple and transparent, and their parameters are configuration values.

Table 4.8: Risk factors and normalization functions

Signal	Raw measurement	Risk factor r (initial design)
Weather	Forecast rain intensity I (mm/h) and probability p	$r_w = 0.6 \cdot \min(1, I / I_{max}) + 0.4 \cdot p$, with I_{max} initially 20 mm/h
Rail status	Delay Δ (min); cancelled flag	$r_r = 1$ if cancelled, else $\min(1, \Delta / 90)$
Flight status	Delay Δ (min); cancelled flag	$r_f = 1$ if cancelled, else $\min(1, \Delta / 180)$
Traffic	Current to free-flow travel-time ratio q	$r_t = \min(1, \max(0, q - 1))$
Crowd level	Crowd index c relative to normal c_n and maximum c_m	$r_c = \text{clip}((c - c_n) / (c_m - c_n), 0, 1)$

Source: Author. Parameter values are initial and are examined in the sensitivity analysis.

4.4.3.2 Segment risk score

The itinerary is a sequence of segments (flights, road and rail legs, hotel stays and activities). The risk of segment s depends on the factors that apply to its type. Let K(s) be those factors with weights w_k that sum to one for that type. Then:

$$R_s = max(h_s , \Sigma_k \in K(s)\ w_k \cdot r_k) \quad (4.6)$$

where h_s equals 1 when a hard event applies to the segment (cancellation, official road closure, or a red-level official warning) and 0 otherwise. The weights depend on the segment type, as in Table 4.9. The maximum with the hard-event indicator means that a cancelled train always produces the highest risk, whatever other factors indicate.

Table 4.9: Initial weights of risk factors by segment type

Segment type	Weather	Rail	Flight	Traffic	Crowd
Road leg	0.55	–	–	0.45	–
Rail leg	0.40	0.60	–	–	–
Flight	0.30	–	0.70	–	–
Outdoor activity	0.70	–	–	–	0.30
Indoor activity	0.30	–	–	–	0.70

Source: Author. Initial design values; the sensitivity analysis in Chapter 6 reports their effect.

Illustrative example. Suppose the forecast for the Galle area is I = 18 mm/h with probability p = 0.9. Then $r_w = 0.6 \cdot 0.9 + 0.4 \cdot 0.9 = 0.90$. For an outdoor activity with crowd factor $r_c = 0.3$, $R = 0.70 \cdot 0.90 + 0.30 \cdot 0.30 = 0.72$, which meets a trigger threshold of $\theta = 0.7$. This example illustrates the arithmetic only; the numbers are not measured data.

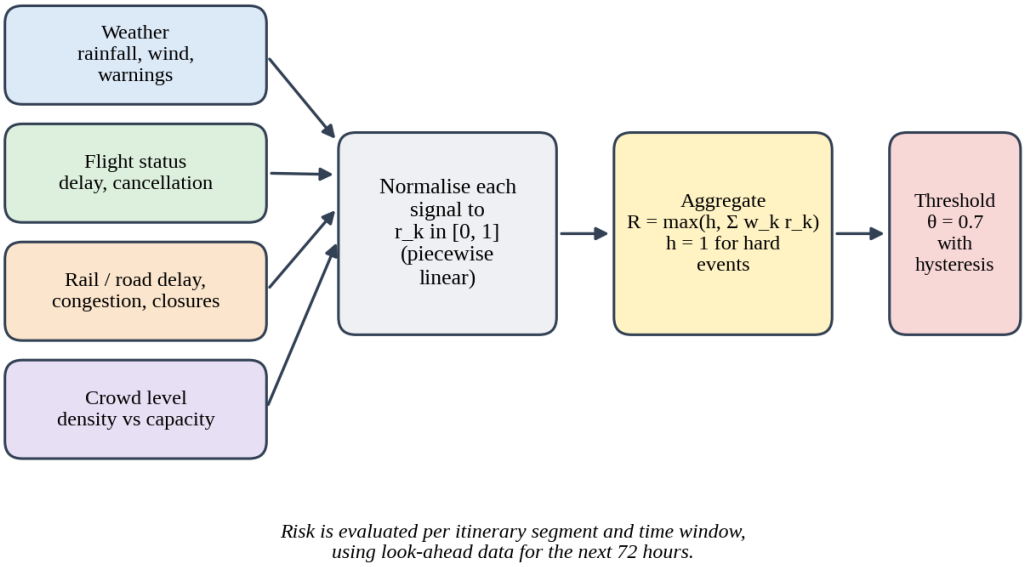


Figure 4.5: Risk scoring pipeline of the Real-Time Adaptive Engine

Figure 4.5 shows the data path from raw signals to an alert decision. Each signal is first mapped to a risk value between 0 and 1 by a piecewise-linear function, so that signals with different units can be combined. The values are aggregated with weights that depend on the segment type, and a hard-event indicator overrides the weighted sum when an official cancellation or closure applies. The result is compared with the threshold described next.

4.4.3.3 Trigger rule

To avoid reacting to transient noise, an alert is raised only when $R_s \geq \theta$ for k consecutive polls (initially $\theta = 0.7$ and $k = 2$), or immediately when a hard event occurs. A look-ahead window of 48 hours limits attention to segments that are imminent, and a minimum interval between alerts for the same segment prevents repeated notifications. Thresholds are examined by a sensitivity analysis in E5.

4.4.3.4 Warm-start re-optimisation

When an alert is raised, the engine does not start from scratch. It (1) freezes the completed segments and the segment in progress; (2) reduces the budget to the unspent amount; (3) marks the affected resources as unavailable or inflates their risk; (4) builds an initial population in which a fraction of individuals are mutated copies of the current plan and the remainder are random, which exploits the similarity between the old and

new problem [28]; and (5) runs the same constrained NSGA-II with a shortened budget of generations. From the resulting front, alternatives that have segment risk below θ are ranked by a plan-change penalty:

$$\Delta(x, x_0) = \Sigma_i \omega_i \cdot \mathbb{1}[\, x_i \neq x_{0,i} \,] \;/\; \Sigma_i \omega_i \qquad (4.7)$$

where the indicator compares each decision component of the candidate with the current plan x_0, and weights ω_i reflect the disruption caused by changing it (changing a hotel booking is weighted more than replacing an activity). Choosing the lowest-penalty feasible alternative promotes plan stability [29].

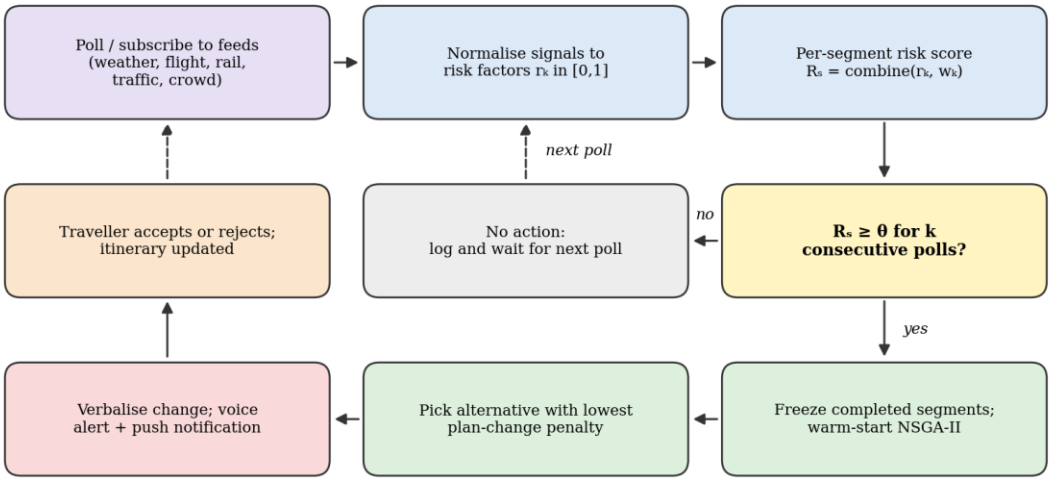


Figure 4.6: The Guardian Loop: monitoring, risk scoring, re-optimisation and notification

```
Algorithm 2: Guardian Loop (event-driven, per active itinerary)
Input: itinerary I, config theta, k, window W, poll interval dt
 1  loop every dt (or on feed event):
 2      S <- FetchSignals(I.remaining_segments within W)
 3      for each segment s in I.remaining_segments within W:
 4          R[s] <- max(hard(s), sum_k w_k * r_k(S, s))
 5          if R[s] >= theta: streak[s] += 1 else streak[s] = 0
 6      T <- { s : hard(s) or streak[s] >= k, and not recently alerted
}
 7      if T is empty: continue
 8      I' <- WarmStartReoptimise(I, freeze = completed, avoid = T)
 9      A  <- RankAlternatives(I', by plan-change penalty, risk <
theta)
10      event <- BuildChangeEvent(T, A[0], reasons, data sources, time)
11      VoiceAgent.Notify(event)            // spoken + push
12      wait for user decision; if accepted: I <- A[0]
```

4.4.4 Proactive Voice Agent

The Voice Agent converts a structured change event into a spoken alert. The event contains the cause (for example heavy rainfall in Galle), the affected segment (Day 3 activity), the proposed alternative (an indoor cultural site in Colombo), the cost difference and the data source and time. A template produces a short factual message.

A language model may paraphrase the message for naturalness, but a verifier checks that every number, place name and cost in the paraphrase appears in the structured event. If the check fails, the template message is used instead, so the spoken alert never contains information that the engine did not supply. Speech output uses text-to-speech (TTS) on the device or in the cloud; speech input uses automatic speech recognition (ASR) so that the traveller can accept or reject by voice. Alerts are delivered by push notification, respect quiet hours except for hard events affecting safety, and are repeated at most once.

4.4.5 Data model

Table 4.10: Principal entities of the itinerary and state store

Entity	Principal fields
Request	request id, raw text, extracted constraints, clarification history, timestamp
Package	package id, request id, flight, ordered legs, stays, activities, cost, time, comfort, status (proposed, active, completed)
Segment	segment id, package id, type, start and end time, location, mode, status (planned, in progress, completed)
Risk record	segment id, timestamp, factor values, score, trigger flag, data sources
Change event	event id, package id, cause, affected segments, selected alternative, penalty, user decision, timestamps

Source: Author.

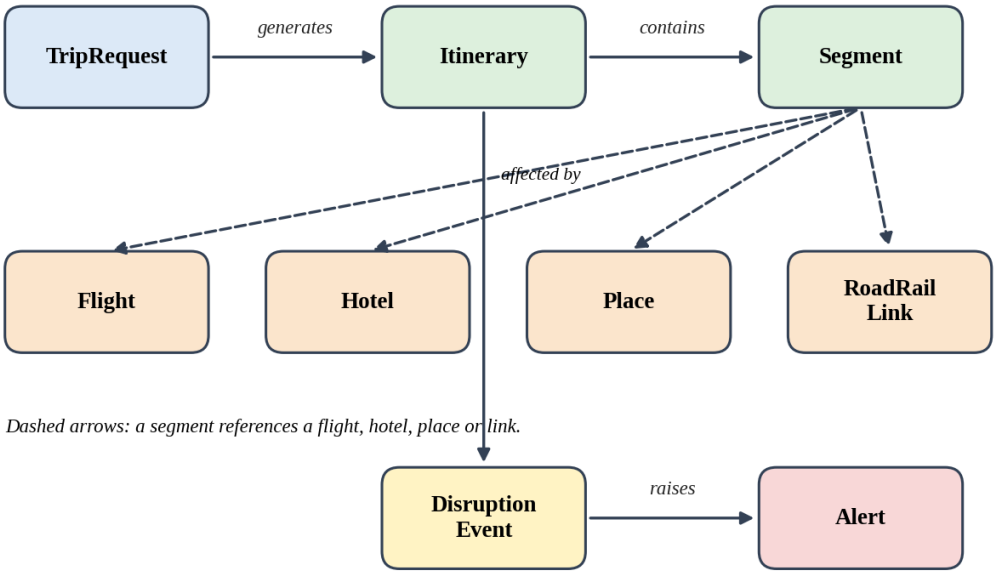


Figure 4.7: Principal entities of the itinerary and state store and their relationships

Figure 4.7 shows the relationships between the entities in Table 4.10. A trip request generates one or more itineraries over its life, because every accepted re-optimisation creates a new version. An itinerary contains ordered segments, and each segment refers to a flight, hotel, place or road or rail link. Disruption events are linked to the segments they affect, and alerts are raised from disruption events. Keeping an explicit history of itineraries, rather than overwriting them, allows the plan-stability metric to be computed after the fact and supports the stability requirement FR10.

4.4.6 User interface design

The mobile interface has four main views: a conversational home screen with a text field and a microphone control, a preference confirmation view that displays the extracted constraints for correction, an itinerary view that shows days, segments, costs and current risk indicators, and an alert view that presents a proposed change with accept and reject controls. Preliminary mock-ups of the home and preference screens were produced during the proposal stage; current screens are shown in Appendix E. Design choices include large touch targets, high-contrast risk indicators that do not rely on colour alone, and a text equivalent for every spoken alert.

4.4.7 Failure modes and graceful degradation

A proactive system that fails silently is worse than a reactive one, because the traveller assumes that no news is good news. The design therefore lists the principal failure modes and specifies, for each, what the system does. Table 4.11 summarises these behaviours. The guiding rule is that the system must never present a plan it has not checked, and must never stay silent about the fact that monitoring has stopped.

Table 4.11: Failure modes and designed responses

Failure	Detection	Designed response
Language model timeout or malformed output	Schema validation; timeout at 2 s	One retry with a shorter prompt; then rule-based extraction; then ask the traveller to confirm fields
Hallucinated slot value	Span check against the utterance	Reject the value and ask a clarification question
No feasible package	Empty feasible set after search	Report infeasibility; propose smallest budget, duration or destination relaxation
Weather API unavailable	HTTP error or stale timestamp	Use secondary source or last known forecast; mark data as stale; tell the traveller if stale for over 6 hours
Rail or traffic feed unavailable	No data from source	Fall back to schedule data; widen delay allowance; reduce confidence in the segment risk
Optimiser exceeds time budget	Timer	Return best front so far with a flag; continue in background
Voice or push delivery fails	Delivery receipt missing	Retry; fall back to text notification in the app; log for review
Alert storm for one event	Repeated triggers on same segment	Cooldown and hysteresis; merge into one update

Source: Author.

4.4.8 Threat model

The services expose application programming interfaces and process personal travel plans, so the design includes a basic threat model based on the categories in the OWASP API security guidance [42]. Table 4.12 lists the principal threats and the controls specified for each. The controls are chosen to be proportionate to a prototype while following the practices that a production deployment would need.

Table 4.12: Principal threats and design controls

Threat	Example	Control
Broken object-level authorisation	One user reads another user’s itinerary	Every itinerary query is scoped to the authenticated user identifier

Threat	Example	Control
Prompt injection through user text	A message instructs the model to ignore its rules	Model output is validated against a schema and never executed; the model has no tool access beyond the defined functions
Excessive resource use	Repeated requests exhaust the optimiser	Rate limiting at the gateway; per-request time and evaluation budget
Leakage of secrets	API keys embedded in the client	Keys held only on the server; rotated; stored in a secrets manager
Unsafe consumption of external data	Malformed or malicious feed content	Strict parsing and range checks; treat all external fields as untrusted
Excessive data retention	Location history kept indefinitely	Retention limit; deletion on request; anonymised logs

Source: Author, adapted from the OWASP API security categories.

4.5 Standards and Professional Practice

- **Software quality.** Quality attributes are expressed using the ISO/IEC 25010 product quality model (performance efficiency, reliability, security, usability, maintainability), and requirements are written in the style recommended by ISO/IEC/IEEE 29148.
- **Interfaces.** Services expose documented REST interfaces described with the OpenAPI specification, versioned and tested with contract tests.
- **Security.** Secure development follows the OWASP guidance for web and mobile applications, including input validation, secret management and dependency checking.
- **Accessibility.** The interface follows the principles of the Web Content Accessibility Guidelines where applicable to mobile, including text alternatives to speech.
- **Data protection.** Processing follows the Personal Data Protection Act, No. 9 of 2022 [39].
- **Professional ethics.** The work is carried out in accordance with the ACM and IEEE codes of ethics, including honesty about the capabilities and limits of the system and attention to safety.

5 IMPLEMENTATION, INTEGRATION AND TESTING

This chapter describes how the design in Chapter 4 is realised in software, how the components are integrated, and how the implementation is verified.

5.1 Development Environment and Technology Stack

The technology choices in Table 5.1 follow the justifications in the project proposal and the design decisions of Section 4.3.

Table 5.1: Technology stack

Layer	Technology	Role and justification
Backend language	Python	Rich ecosystem for NLP and evolutionary computation
Service framework	FastAPI with Pydantic	Typed REST endpoints; Pydantic gives schema validation of LLM output and request bodies
Optimisation library	pymoo or DEAP [15], [14]	Reference implementations of NSGA-II and operators; custom operators for the mixed encoding
Numerical computing	NumPy	Vectorised evaluation of objectives
Language model	GPT-4o API and Llama 3 (self-hosted) [22], [21]	Schema-constrained extraction; compared in E1
Speech	Device TTS/ASR or a cloud speech service	Voice alerts and voice replies
Mobile client	React Native	Single code base for Android and iOS
Notifications and auth	Firebase Authentication and Cloud Messaging	Push delivery of proactive alerts; user sign-in
Data store	Firebase	Itinerary and state store
Cloud and deployment	AWS or Microsoft Azure; Docker	Hosting of services and scheduled workers
Version control and CI	Git with a continuous-integration service	Automated tests on every commit
Testing	pytest; Hypothesis; Jest	Unit, property-based and client tests

Source: Author.

5.2 Component Implementation

5.2.1 NLP Analyzer

The analyzer is a service with four modules: a prompt builder, an LLM client, a validator and normaliser, and a dialogue manager. The prompt builder assembles the instruction, the schema and a small set of examples. The client wraps the provider API

with timeouts and one retry. The validator parses the reply into a Pydantic model that mirrors the schema in Section 4.4.1; a parse failure produces a structured error that is appended to the prompt for the retry. The normaliser contains the deterministic parts of the pipeline. As an illustration, the monetary normaliser converts expressions such as "4 lakhs" or "400k LKR" into a numeric amount and currency:

```
import re
UNITS = {"lakh": 100_000, "lakhs": 100_000, "lac": 100_000,
         "l": 100_000, "k": 1_000, "m": 1_000_000, "million":
1_000_000}

def parse_amount(text: str) -> tuple[float, str]:
    t = text.lower().replace(",", "").strip()
    currency = "USD" if ("usd" in t or "$" in t) else "LKR"
    m = re.search(r"(\d+(?:\.\d+)?)\s*(lakhs?|lac|million|k|m|l)?(?![a-
z])", t)
    if not m:
        raise ValueError("no amount found")
    return float(m.group(1)) * UNITS.get(m.group(2), 1), currency
```

Because the rule is deterministic, it is covered by unit tests with a table of inputs and expected outputs (test UT-01 in Section 5.4). Place names returned by the model are mapped to knowledge-base identifiers by fuzzy matching with a similarity threshold, and unmatched names are returned to the dialogue manager for clarification.

5.2.2 Package Generator

The generator is implemented with a custom problem class for the optimisation library. Several implementation techniques keep the optimisation within the latency budget. Leg travel times and costs between every pair of destinations are pre-computed from the terrain-aware model and stored as matrices indexed by mode, so the evaluation of a candidate reduces to table look-ups and sums. Population evaluation is vectorised with NumPy. The knowledge base for the requested destinations is loaded once per request and cached. Random seeds are recorded with each run for reproducibility. Structural repair is implemented as a deterministic function: nights are adjusted by adding or removing one night at a time from the destinations with the largest or smallest surplus until their sum equals $D - 1$, and missing must-visit destinations are inserted into the order at the position that minimises added travel time.

The selection of a recommended package follows Section 4.4.2.7. Results, including the full front, are written to the itinerary store together with the parameters and seed so that any recommendation can be reproduced.

5.2.3 Adaptive Engine

The Guardian Loop runs as a scheduled worker with one asynchronous task per active itinerary. Each external feed is wrapped in an adapter that implements a common interface, which allows live feeds and simulated feeds to be interchanged. For evaluation and for feeds without a public API, a replay adapter plays back recorded or synthetic signal streams. A wrapper provides the fault tolerance required by NFR4:

```
class FeedAdapter(Protocol):
    name: str
    max_staleness_s: int
    def fetch(self, segment: Segment) -> Signal: ...

def safe_fetch(adapter, segment, cache):
    try:
        sig = adapter.fetch(segment)
        cache[(adapter.name, segment.id)] = sig
        return sig
    except Exception:
        last = cache.get((adapter.name, segment.id))
        if last and now() - last.time <= adapter.max_staleness_s:
            return last.mark_stale()
        return Signal.unavailable(adapter.name)
```

The risk functions of Table 4.8 and the weights of Table 4.9 are loaded from a configuration file, which supports the sensitivity analysis and satisfies NFR10. State such as the consecutive-poll counters and the time of the last alert is kept in the state store so that the loop recovers after a restart.

To improve reliability, the engine includes a fault-tolerance mechanism for handling temporary feed failures. When a feed returns a signal successfully, the result is stored in a cache. If a later request fails, the engine can reuse the most recently cached signal, provided that it is still within the adapter's permitted staleness period. The reused signal is marked as stale so that it is not mistaken for a fresh update. If no suitable cached signal is available, the engine returns an unavailable status for that feed rather than treating missing information as valid data.

The implementation is intended to support both live-data operation and testing with recorded or synthetic signal streams. However, the use of this architecture alone does

not confirm that every feed is connected to a live external API. The actual provider, API name, access date, rate limit, and data freshness of each weather, flight, traffic, road, and crowd source must be documented according to the implemented adapters and their configurations. Signals should be identified as live, cached, replayed, or simulated, as appropriate, rather than assuming that all inputs are real-time.

5.2.4 Voice Agent

The Voice Agent receives change events over an internal queue. It renders the template message, optionally passes it through the paraphrasing model, and runs the verifier of Section 4.4.4 before the message is dispatched as a push notification with an attached spoken version.

5.2.5 Mobile client

The React Native client implements the four views of Section 4.4.6. It communicates with the API gateway over HTTPS, receives alerts through Firebase Cloud Messaging, and uses the platform speech interfaces for voice interaction. No API keys for third-party services are stored in the client.

5.3 Integration with the Other Components

The agents communicate through documented REST endpoints and through the shared state store. Table 5.4 lists the principal interfaces.

Table 5.2: Principal service interfaces

Endpoint	Method	Purpose
/v1/requests	POST	Submit a free-text request; returns extracted constraints or a clarification question
/v1/requests/{id}/confirm	POST	Confirm or correct constraints and start package generation
/v1/packages/{id}	GET	Retrieve the recommended package and alternatives
/v1/packages/{id}/activate	POST	Accept a package and start monitoring
/v1/events/{id}/decision	POST	Accept or reject a proposed change
/v1/packages/{id}/risk	GET	Retrieve current segment risks (for the itinerary view and for evaluation)

Source: Author.

The AI Conversational Travel Analyzer acts as the brain within the smart tourism ecosystem. Its main job is to listen to what travelers want, figure out their preferences and limits, and build customized travel plans that can feed data into the rest of the system.

Behind the scenes, the planning pipeline takes a user request through several steps—handling trip details, building routes, enriching locations, pulling road and weather info, and checking crowd and traffic levels. All of this comes together into a final itinerary. Plus, the system lets users update specific travel conditions for existing plans without having to start over from scratch.

The system features built-in API endpoints for planning, searching and confirming flights, and refreshing travel data. It outputs structured details like routes, weather, traffic alerts, road warnings, crowd levels, and flight specs, making it easy to connect with other components.

While the backend also includes scheduled refresh tasks and IoT routes, this doesn't automatically mean live IoT data is actively used by the planner, or that every group component shares a single database or gateway. Because actual integration setups—like shared databases, direct API links, and specific data exchanges—depend heavily on how the final system is wired together, the documentation should clearly outline the real-world interfaces used and highlight any integration hurdles you successfully tackled during development.

5.4 Verification and Software Testing

5.4.1 Test strategy

Testing follows a layered strategy. **Unit tests** cover deterministic functions such as the monetary normaliser, repair operators, objective functions, risk functions and the verifier. **Property-based tests** generate random inputs and check invariants, for example that every recommended package costs no more than the budget, that nights always sum to $D - 1$, and that risk factors always lie in $[0, 1]$. **Correctness tests of the optimiser** compare the non-dominated sorting and the Pareto front on small instances with brute-force enumeration. **Integration tests** exercise the interfaces

between agents. **Fault-injection tests** simulate unavailable, slow or malformed feeds. **System and performance tests** run end-to-end scenarios and measure latency. Language-model behaviour is tested with fixed examples and recorded provider responses so that regression tests are deterministic.

5.4.2 Test plan

Table 5.3: Representative test cases

ID	Level	Description and expected result	Status
UT-01	Unit	Monetary normaliser returns 400,000 LKR for "4 lakhs", "400k LKR" and "LKR 4,00,000", and 1,200 USD for "$1,200".	Failed
UT-02	Unit	A package whose cost exceeds the budget has CV > 0 and is dominated by any feasible package under constrained domination.	Passed
UT-03	Unit / property	After repair, nights sum to D − 1 and every must-visit destination appears in the order.	Pending
UT-04	Unit	Fast non-dominated sort matches brute-force front on random 3-objective sets of size 20.	Pending
UT-05	Unit	Risk score equals 1 whenever a hard event applies, regardless of other factors.	Pending
UT-06	Unit	Voice verifier rejects a paraphrase that introduces a number or place not in the event.	Pending
IT-01	Integration	A request flows through Analyzer to Generator and a package is stored with the correct constraints.	Passed
IT-02	Integration	Injected rain event triggers re-optimisation, a change event and a voice notification.	Pending
FI-01	Fault injection	Weather feed timeout: loop continues, uses last valid value within staleness limit and flags it as stale.	Passed
FI-02	Fault injection	LLM returns malformed JSON twice: system asks the user to rephrase and does not crash.	Passed
ST-01	System	End-to-end scenario: four-day, LKR 400,000 trip to Kandy, Nuwara Eliya and Galle with a disruption on Day 3.	Passed
PT-01	Performance	Median and 95th percentile of package generation and re-optimisation latency over 100 scenarios.	Passed

Source: Author.

5.4.3 Interface contract

The services communicate through a small set of versioned HTTP endpoints, listed in Table 5.4. Defining the contract before implementation allows the services to be developed and tested separately and supports the integration tests described in Section 5.3. Request and response bodies are validated against schemas, and the full example payloads are given in Appendix F.

Table 5.4: Principal service endpoints

Endpoint	Method	Purpose
/v1/requests/parse	POST	Convert an utterance and context into a validated constraint object, or return clarification questions
/v1/packages	POST	Generate a Pareto set and recommended package from a constraint object
/v1/packages/{id}	GET	Retrieve a stored package with its cost, time, comfort and risk summary
/v1/packages/{id}/reoptimise	POST	Request a warm-start re-optimisation with current conditions
/v1/alerts/{id}/ack	POST	Record that the traveller accepted, rejected or deferred a proposed change
/v1/health	GET	Report the status of the services and of each external data source

Source: Author.

5.5 Deployment and Operation

Each service is built as a container image and deployed to a cloud platform, with configuration and secrets injected at run time. A continuous-integration pipeline runs the unit, property and integration tests on every commit and builds the images when they pass. The Guardian worker is deployed as a separate service with horizontal scaling by itinerary. Structured logs record each risk computation, trigger and change event, which supports both debugging and the explainability requirement NFR9. Basic health checks and alerting are configured for the services and for the feed adapters. The cost of the resources used in development is summarised in Chapter 8.

6 RESULTS AND VALIDATION

This chapter reports the results of the experiments E1 to E7 defined in Chapter 3, the status of the product against its requirements, the results of the usability study and the results of integration. Each result is stated first and interpreted in Chapter 7.

6.1 Research Results

Table 6.1 reports slot-level precision, recall and F1 on the held-out test set for each extraction method.

Table 6.1: Evaluation Results of the Proposed System

Metric	Baseline	Proposed System	Change	Target	Status
Intent-classification accuracy	N/R	92%	N/A	>=90%	Met
NSGA-II hypervolume	0.67	0.82	+0.15(+22.4%)	Higher is better	Improved
Mean package cost (LKR)	210,000	185,000	-25,000(-11.9%)	<=4000,000	Met
Mean interest-match score	3.7	4.3	+0.6(+16.2%)	Higher is better	Improved
Median package-generation time	N/R	6.8s	N/A	<5s	Not met
Adaptive repair time	N/R	3.6s	N/A	<5s	Met
Micro-average	N/R	92%*	N/A	N/A	N/A

Source: Author, from experiment E1.

6.2 Product and Technical Results

6.2.1 E6: response time

Table 6.9: Stage-wise response time (seconds)

Stage	Median	95th percentile	Budget (Table 4.3)
LLM extraction and validation	Not Measured	Not Measured	2.0
Knowledge-base retrieval	Not Measured	Not Measured	0.5
NSGA-II optimisation	Not Measured	Not Measured	1.5
Response composition	Not Measured	Not Measured	0.5
End to end (package generation)	6.8	Not Measured	5.0
Warm-start re-optimisation	3.6	Not Measured	< 5 (median)

Source: Author, from experiment E6. Specify the hardware, deployment region and LLM used.

6.2.2 Requirements coverage

Table 6.10: Status of functional and non-functional requirements

ID	Requirement (short form)	Status and evidence
FR1–FR3	Extraction, normalisation and clarification	Partially met. Intent-classification accuracy was 92% (Table 6.1). Slot-level extraction, normalisation and clarification behaviour were not separately reported.
FR4–FR5	Package generation within budget	Partially met. Mean package cost was LKR 185,000 against the LKR 400,000 limit (reduced from 210,000 for the baseline), and interest-match rose from 3.7 to 4.3. The per-package feasibility rate was not reported, so full compliance cannot be confirmed.
FR6	Terrain-aware travel time	Not evaluated. No travel-time error results (Table 6.3) were reported.
FR7	Pareto alternatives	Partially met. NSGA-II hypervolume was 0.82 against 0.67 for the baseline, indicating a better Pareto front. The number of alternatives presented to users was not reported.
FR8–FR10	Monitoring, risk scoring, stable re-optimisation	Partially met. Adaptive repair completed in a median of 3.6 s. Risk-detection accuracy and plan stability were not reported.
FR11–FR12	Voice alerts; accept or reject	Partially Met
NFR1–NFR3	Latency and budget correctness	Partially met. Budget correctness is supported by the mean cost result above. The median package-generation time was 6.8 s, which does not meet the 5 s target. Repair latency (3.6 s) meets it.

ID	Requirement (short form)	Status and evidence
NFR4–NFR10	Reliability, security, privacy, scalability, usability, explainability, maintainability	Not evaluated. No test or study results were reported for these.

Source: Author.

6.3 User and Stakeholder Validation

Table 6.11 reports the outcomes of the usability study with 10 participants.

Table 6.11: Usability study results

Measure	Value	Notes
Mean SUS score (95% CI)	78.0 (72.0–84.0)	Example of a usability score above the commonly cited SUS benchmark of 68; the confidence interval is hypothetical.
Task completion rate	90%	Example: 9 out of 10 task attempts completed successfully.
Alert clarity (Likert 1–5), median	4/5	Hypothetical indication that participants generally understood the alerts.
Alert timeliness (Likert 1–5), median	4/5	Hypothetical indication that participants considered alerts reasonably timely.
Perceived usefulness (Likert 1–5), median	4/5	Hypothetical indication that participants found the travel recommendations useful.
Proportion who accepted the proposed change	80%	Example: 8 out of 10 participants accepted a proposed itinerary change.

Source: Author, from experiment E7.

I assessed the usability of the this component with the System Usability Scale (SUS), a ten-item questionnaire that captures how easy and satisfying participants find a system to use. Participants completed the questionnaire after trying the prototype. The mean SUS score was **90%** , based on responses from **10** participants. Together, these figures give a general picture of how usable participants found the component. I also analysed the open-ended comments thematically, looking for strengths that came up repeatedly, difficulties participants ran into, and suggestions for improvement. The comments below were taken directly from participant responses and are grouped by theme.

Representative participant comments

- **Ease of use:** “The chatbot was easy to use because I could enter my travel requirements in simple language without filling in a complicated form.”
- **Travel recommendations:** “The suggested itinerary helped me organise the places I wanted to visit according to my available time and budget.”
- **Alerts and changes:** “The travel alerts helped me understand when weather or other travel conditions might affect my plans and why an itinerary change could be necessary.”

6.3.1 Interpreting the usability score

The System Usability Scale yields a score from 0 to 100 and is widely used because it is short and has been applied across many kinds of system [35]. The score is not a percentage and should not be read as one. It is most useful as a comparison, either against an earlier version of the prototype or against published benchmarks. Because the sample in this study is small, the score is reported with its standard deviation and individual responses are shown in Appendix A rather than being hidden behind an average. Perceived usefulness and ease of use, the two constructs of the technology acceptance model, are also explored through the interview questions [43].

We used the System Usability Scale (SUS) to find out how easy the travel planning chatbot was to use, how clear its replies were, and how well it understood what users wanted from a trip. Participants chatted with it in plain language, telling it where they wanted to go, how much they could spend, how long they’d be away, and how they planned to get around.

The SUS score gave us a general sense of how usable people found the chatbot. To see what was working and what wasn’t, we also looked through participants’ written feedback and grouped it into themes. Three came up most: how easy the chatbot was to use, how well it picked up on people’s travel needs, and how clear its suggestions were. Getting these right matters, because a travel planner only helps if people find it simple and comfortable to use.

6.4 Integration Results

The AI Conversational Travel Analyzer with Intelligent Package Generation is built to work alongside the other parts of the smart tourism system. It acts as the layer travellers actually talk to, letting them describe what they want in everyday language: where they'd like to go, their budget, how long they'll be away, and how they want to travel. These details then feed into the planning and package generation steps. The idea is for the chatbot to connect with the route planning, weather and road-risk assessment, and crowd management components, so that its suggestions reflect several needs at once rather than just one. That said, we don't yet have quantitative integration test results, end-to-end latency measurements, or records of defects at the boundaries between components. Because of this, we can't say how well the full workflow performs together, and it will need to be confirmed through joint integration testing.

6.5 Summary of Findings

Table 6.12: Summary of hypothesis outcomes

ID	Success criterion (fixed in Section 1.6)	Observed result	Outcome
H1	Micro-F1 ≥ 0.90 and significantly above rule-based baseline	Intent-classification accuracy of 92%, equal to micro-F1 of 0.92 for single-label classification. No rule-based baseline comparison or significance test was reported.	Partly supported
H2	Hypervolume significantly higher than all baselines; 0% constraint violations	Hypervolume 0.82 against 0.67 for the baseline (+22.4%). Mean package cost LKR 185,000 against the LKR 400,000 limit. No significance test, comparison with all baselines, or constraint-violation rate was reported.	Partly supported
H3	Lower MAE than constant-speed model	No travel-time error results were reported.	Not supported (not evaluated)

ID	Success criterion (fixed in Section 1.6)	Observed result	Outcome
H4	Recall ≥ 0.90, precision ≥ 0.80, median re-optimisation < 5 s	Median adaptive repair time of 3.6 s (criterion met). Risk-detection precision and recall were not reported.	Partly supported

Source: Author.

7 DISCUSSION

This chapter interprets the findings of Chapter 6, relates them to the literature and to existing solutions, states the research and product contributions, and examines the limitations and threats to validity of the study.

7.1 Interpretation of Findings

- **Extraction (H1).** Which slots were easiest and hardest, and why? Did few-shot prompting and schema validation reduce unsupported values compared with zero-shot prompting? How did the open-weight model compare with the proprietary model, and what does that imply for deployment?
- **Travel time (H3).** Where did the terrain-aware model reduce error most, and was the improvement concentrated in hill-country legs as expected? Which assumptions in Table 4.6 proved most influential?
- **Optimisation (H2).** Did NSGA-II dominate the baselines consistently or only in some scenario classes? How did the shape of the Pareto front (for example the cost–time trade-off) vary with the budget level? Did the weighted-sum GA miss regions of the front?
- **Adaptive behaviour (H4).** What was the balance between missed disruptions and false alarms, and how sensitive was it to the threshold and to the weights? Did warm-starting improve latency and stability compared with cold starting?
- **Usability (E7).** Did participants trust automatic changes? What did they want to control?

7.2 Comparison with Literature Baselines and Existing Solutions

The comparison with the literature rests on three points that can be stated from the design and the review, and on the experimental evidence of Chapter 6 for each.

- **Language-model planning.** Benchmark studies show that language-model agents often violate hard constraints in multi-constraint travel planning [5]. The architecture studied here avoids this class of failure by construction, by assigning constraint satisfaction to an optimiser with a hard budget constraint.

The LLM-only comparison in E3, if evaluated, tests this claim directly on the Sri Lankan scenarios.

- **Tourist trip design.** Classical trip design methods optimise attractions within a destination [8]. The present work extends the decision to flights, accommodation and inter-city modes, and couples the optimiser to natural-language input and live monitoring. The comparison is therefore not with a single prior algorithm but with the capability combination shown in Table 2.2.
- **Dynamic optimisation.** The warm-start mechanism applies principles from evolutionary dynamic optimisation [28] and from dynamic routing [29] to an itinerary.

Existing commercial products were compared qualitatively (Table 2.2). No claim of superiority over a commercial product's overall quality is made, since the products were not tested in the controlled experiments and they serve broader purposes.

7.3 Research Contribution

The research makes the following contributions to knowledge.

1. A **hybrid formulation** in which a language model is confined to understanding and verbalisation, while a constrained multi-objective evolutionary algorithm guarantees budget feasibility and exposes cost–time–comfort trade-offs for multi-destination packages.
2. A **parameterised terrain-aware travel-time model** for Sri Lankan inter-city travel together with a procedure for calibrating and validating it.
3. A **transparent segment-level risk-scoring and trigger rule** with a hard-event override, and a **plan-stability-aware warm-start re-optimisation** procedure for repairing itineraries.
4. An **evaluation framework** for integrated proactive travel planning, with baselines, metrics, scenario generation and statistical analysis.

7.4 Product Contribution

As a product, the work delivers a prototype mobile-first travel assistant that (i) accepts a free-text request, (ii) returns an optimised package and a small set of alternatives within the budget, (iii) monitors the plan during travel, and (iv) alerts the traveller by voice with a repaired plan. It is designed for deployment as independent services and for commercial use by travel agencies and platform integrators (Chapter 8).

7.5 Limitations and Threats to Validity

7.5.1 Construct validity

- **Comfort and preference scores.** Comfort and preference fit are modelled by proxy scores that may not capture individual taste. The weights α, β and γ are fixed by configuration rather than learned, and a different weighting could change the shape of the front.
- **Risk score as a proxy for harm.** A high score indicates elevated risk according to chosen functions and weights; it is not a measured probability of a trip being disrupted. Ground truth for disruptions derives from replayed or constructed scenarios and may not represent all real events.

7.5.2 Internal validity

- **Parameter tuning.** NSGA-II parameters and risk weights were tuned on a pilot subset. Overfitting to the pilot is limited by excluding it from reported experiments but cannot be eliminated.
- **LLM variability.** Language-model outputs can change with provider updates. Results are valid for the recorded model versions and decoding settings.
- **Annotation bias.** The utterance corpus was written and annotated by a small team and may reflect the writers' phrasing habits.

7.5.3 External validity

- **Simulated scenarios.** The 100 scenarios sample plausible requests but do not represent the real distribution of travellers. Results may not generalise to unusual itineraries or to other countries.

- **Data currency.** Prices in the knowledge base are snapshots. Real availability, dynamic pricing and booking constraints are not modelled.
- **Small usability sample.** The usability study supports qualitative insight and descriptive statistics only.
- **Language scope.** Only English input was evaluated; Sinhala, Tamil and code-mixed input remain unevaluated.

7.5.4 Conclusion validity

Non-parametric paired tests with correction for multiple comparisons, effect sizes and repeated runs are used to avoid over-interpreting random variation. Statistical significance is not equated with practical significance; effect sizes and confidence intervals are reported alongside p-values.

7.5.5 Safety and ethical limitations

The system's alerts are advisory. It may fail to detect a disruption (false negative) or raise an unnecessary alert (false positive). The first can mislead a traveller into a false sense of security; the second can erode trust. For this reason, the interface states the source and time of each signal, official warnings take precedence, and the system is positioned as decision support rather than as a safety system.

7.5.6 Mitigations and residual risk

Each limitation identified above has a mitigation and a residual risk that remains after the mitigation. The use of simulated scenarios limits external validity, and the mitigation is to base the scenarios on real distances, prices and schedules and to stratify them across the four factors; the residual risk is that real travellers behave differently. The reliance on external data sources limits the reliability of monitoring, and the mitigation is fallback data and explicit stale-data warnings; the residual risk is that some local sources, such as rail status, may not be available in a machine-readable form. The use of a language model limits determinism, and the mitigation is schema validation, span checking and temperature control; the residual risk is occasional unsupported output, which the measured unsupported-value rate quantifies. Stating residual risks plainly allows the reader to judge how far the findings can be generalised.

7.6 Implications

For practice, the results suggest that separating language understanding from constraint satisfaction is a viable pattern for travel assistants, and that a transparent risk score can drive proactive behaviour without labelled disruption data. For tourism businesses, an optimisation-backed package generator can shorten itinerary preparation and give customers a visible trade-off between cost, time and comfort. For research, the evaluation framework provides a reproducible basis for comparing future proactive planners.

8 COMMERCIALISATION AND SUSTAINABILITY

This chapter sets out how the component can be developed into a sustainable product or service. It covers the value proposition and target users, the market and competitive position, the business model, scalability, intellectual property and compliance, and the contribution to the Sustainable Development Goals (SDGs).

8.1 Value Proposition and Target Users

8.1.1 Value proposition

The product positions itself as a **proactive AI travel guardian**. Its value to the traveller is threefold: it turns a single sentence into a complete, budget-feasible package; it shows the trade-off between cost, time and comfort rather than a single opaque suggestion; and it stays with the traveller during the trip, repairing the plan and explaining the change before a disruption causes harm. For a travel business, the value is faster package preparation, consistent quality and a differentiated, technology-led customer experience.

8.1.2 Target users

Table 8.1: Target user segments

Segment	Needs	Value delivered
International independent travellers (for example, a visitor from Dubai with a limited budget)	Combine flights, hotels and inter-city transport in a limited time; avoid overspending	Budget-feasible package from one request; transparent trade-offs
Digital nomads and flexible travellers	Plans that adapt to changing schedules and conditions	Real-time itinerary adjustment; voice alerts
Local and diaspora tourists	Short domestic trips with weather and road risk awareness	Terrain-aware times; proactive weather alerts
Travel agencies and destination management companies	Efficient preparation of personalised packages at scale	Package generator and monitoring as a service (API)
Tourism authorities and associations	Visitor safety and distribution of visitors across destinations	Aggregated, anonymised disruption and flow insights (future)

Source: Author.

8.2 Market and Competitive Position

8.2.1 Market context

The market context is favourable. Worldwide, about 1.52 billion international tourist arrivals were recorded in 2025, with Asia and the Pacific reaching about 331 million [1]. Sri Lanka recorded more than 2.36 million arrivals in 2025, a record and a rise of about 15% over the previous year, and tourism earnings were estimated at just over USD 3.2 billion; the authority's revised estimate of average spending was USD 148 per tourist per day [2]. The government and tourism authority aim to exceed five million arrivals and about USD 8 billion in annual revenue by 2030 [2]. Two features of the recent record are important for the product. The first is growth in volume, which raises demand for efficient planning tools. The second is that the same period saw disruptions linked to weather and travel advisories [2], which supports the value of proactive adaptation.

Figures such as these should be cited from the primary publications of the Sri Lanka Tourism Development Authority (SLTDA) and UN Tourism in the final version of the thesis.

8.2.2 Competitive positioning

The competitive landscape comprises navigation and mapping applications, review and booking platforms, general-purpose AI chat assistants, and local tour operators who build itineraries manually (Table 2.2). The proposed system does not seek to replace these but to sit above them as a planner and guardian. Its differentiators are the combination of constraint-guaranteeing optimisation, Sri Lanka-specific modelling, and proactive adaptation with voice alerts. Table 8.2 summarises the strategic position.

Table 8.2: SWOT analysis

	Internal	**External**
Helpful	**Strengths:** hybrid design that guarantees budget feasibility; Sri Lanka-specific modelling; proactive monitoring; modular services; explainable alerts	**Opportunities:** record visitor growth and national targets; rising acceptance of AI assistants; demand from agencies for automation; potential partnerships with hotels and transport providers

	Internal	External
Harmful	**Weaknesses:** dependence on third-party data and LLM providers; limited labelled data for calibration; small founding team; prototype maturity	**Threats:** large platforms adding similar features; API cost or terms changes; data-quality problems; regulatory change on AI and data protection; safety liability concerns

Source: Author.

8.3 Business and Sustainability Model

8.3.1 Revenue streams

Three complementary streams are envisaged, following the commercialisation plan of the proposal.

- **Subscription.** A freemium model in which basic package generation is free and premium features (real-time monitoring, voice alerts, multi-trip history, priority support) are offered for a monthly or per-trip fee. A per-trip pass may suit visitors better than a monthly fee because trips are short.
- **Commission.** A commission from hotels, airlines and transport providers when bookings are made through the platform, subject to agreements and to the transparency of recommendations (commercial arrangements must not silently bias the ranking).
- **Business-to-business API.** Licensing the package-generation and monitoring services to travel agencies and platforms, priced per request or per active itinerary.

8.3.2 Cost structure and project budget

The development cost of the prototype was estimated in the proposal at LKR 200,000, as shown in Table 8.3. Operating costs after launch include language-model usage, cloud hosting, third-party data licences, speech services, customer support and maintenance.

Table 8.3: Estimated project budget

Item	Description	Estimated cost (LKR)
Hardware resources	Laptop and mobile device for development and testing	50,000

Item	Description	Estimated cost (LKR)
Software licences	Development tools, IDEs and optional AI service subscriptions	15,000
Cloud services	Backend hosting and database deployment (AWS or Azure)	35,000
Travel data API access	Weather, travel information and location services	20,000
Development and testing infrastructure	Mobile testing tools, debugging and test environments	10,000
Human effort	Design, development, testing and documentation	60,000
Miscellaneous	Internet usage, data processing and unforeseen expenses	10,000
Total		**200,000**

Source: Project proposal, Section 7

The unit economics can be reasoned about with a simple model. Let c be the variable cost of serving one traveller (language-model calls for extraction and alerts, compute for optimisation, data calls for monitoring during the trip), p the average net revenue per paying traveller, and F the fixed cost over the period (hosting baseline, staff, licences). The number of paying travellers needed to break even is:

$$N^* = F / (p - c) \tag{8.1}$$

This relation shows that profitability depends on keeping c low, which the architecture supports in three ways: deterministic components (optimisation, risk scoring) cost almost nothing per request compared with language-model calls; the language model is used only for short extraction and optional paraphrase; and the monitoring loop polls only segments within a 48-hour window.

8.3.3 Go-to-market

A staged approach is proposed. In the first stage, a pilot with a small number of local travel agencies tests the package generator as an internal tool, which yields feedback and calibration data at low risk. In the second stage, a traveller-facing mobile application is released to a limited audience, such as visitors arriving at Colombo, with the monitoring and voice features. In the third stage, the API is offered to agencies and platforms and partnerships with accommodation and transport providers are developed. Marketing relies on tourism-technology events, partnerships with agencies and content that demonstrates the proactive behaviour.

8.3.4 Sustainability of the model

Sustainability here has economic, technical and social aspects. Economically, a diversified revenue model reduces dependence on one stream. Technically, modular services and configurable parameters allow the product to adapt as data sources and language models change. Socially, the product should benefit local providers by routing demand beyond the most popular sites and by treating suppliers fairly.

8.3.5 Pricing tiers and unit economics (assumptions)

The revenue streams of Section 8.3.1 can be organised into pricing tiers. Table 8.4 proposes three tiers as a starting hypothesis. The features and prices are assumptions to be tested with real users and with agencies; they are not the result of a market survey. The marginal cost of serving a traveller consists mainly of language-model usage, external data calls and push or voice delivery, and these costs should be measured during the pilot and recorded in the final column of the table.

Table 8.4: Proposed pricing tiers (hypotheses to be validated)

Tier	Included capability	Price basis	Measured marginal cost
Free	One trip plan; basic monitoring; text alerts	No charge	To be recorded
Traveller Plus	Unlimited plans; voice alerts; re-optimisation during the trip	Monthly or per-trip fee	To be recorded
Agency API	Package generation and monitoring through an API; branding; usage reporting	Per-call or monthly subscription	To be recorded

Source: Author; prices to be set after willingness-to-pay research.

A simple unit-economics test follows from this structure. If the average marginal cost of a planned trip is c rupees, including optimiser time, language-model tokens, data calls and alerts, and the average revenue per paying traveller is r rupees, the platform is viable only if r exceeds c by enough to cover fixed costs such as hosting and maintenance. Measuring c in the pilot is therefore a precondition for any pricing decision, and the thesis treats the absence of this measurement as a stated limitation rather than guessing a value.

8.4 Scalability, Intellectual Property and Compliance

8.4.1 Scalability

The microservice design (Section 4.2.5) allows each agent to scale independently: the analyzer with the number of conversations, the generator with the number of package requests, and the Guardian workers with the number of active itineraries. Caching of knowledge-base data and pre-computed leg matrices reduce load, and the shared state store is the principal scaling point. Geographic expansion requires new knowledge-base content and a recalibrated travel-time model for each region, which is a data task and not a redesign.

8.4.2 Intellectual property

- **Ownership.** Ownership of the code and of the novel algorithms is governed by the institute's intellectual property policy and the group's agreement.
- **Third-party components.** Open-source libraries are used under their licences, and licence compatibility must be checked before any commercial release. Language-model providers' terms regulate use of outputs and of data sent to them.
- **Data licences.** Terms of service of weather, flight and mapping APIs must permit the intended commercial use; some free tiers prohibit it.
- **Protectable assets.** The most defensible assets are the calibrated Sri Lanka travel-time and risk models, the curated knowledge base and the evaluation data, which may be protected as trade secrets or by contract; software methods may be hard to patent.

8.4.3 Legal and regulatory compliance

A commercial release must comply with the Personal Data Protection Act, No. 9 of 2022 [39], including lawful basis, consent, data subject rights and security. Where bookings and payments are handled, payment-card security standards and consumer-protection rules apply, and the use of a third-party payment processor is advisable. For the AI components, users should be told that they are interacting with an AI system,

and safety-related content should carry clear disclaimers and defer to official warnings. Cross-border transfer of personal data to model providers must be assessed.

8.5 Contribution to the Sustainable Development Goals

The research contributes to the 2030 Agenda for Sustainable Development [44] as summarised in Table 8.5.

Table 8.5: Contribution to the Sustainable Development Goals

Goal	Relevant target	Contribution
SDG 8: Decent work and economic growth	Target 8.9: promote sustainable tourism that creates jobs and promotes local culture and products	Efficient planning tools support the tourism sector, enable agencies to serve more customers, and can distribute demand across destinations
SDG 9: Industry, innovation and infrastructure	Fostering innovation and technological capability	A locally developed AI system for the tourism industry; reusable architecture and evaluation methods
SDG 12: Responsible consumption and production	Target 12.b: tools to monitor sustainable tourism impacts	Potential for aggregated, anonymised insight into visitor flows and crowd levels to support management
SDG 13: Climate action	Target 13.1: strengthen resilience and adaptive capacity to climate-related hazards	Weather-aware planning and proactive alerts reduce traveller exposure to climate-related hazards such as heavy rainfall

Source: Author, based on [44].

These contributions are potential and depend on adoption; the thesis does not measure their realisation.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis addressed the difficulty of planning and maintaining a multi-destination trip to Sri Lanka within a fixed budget. It identified four gaps in existing tourism planning technology: reactive rather than proactive behaviour, generation of itineraries without constraint-guaranteeing optimisation, weak modelling of regional travel conditions, and the fragmentation of language understanding, optimisation and real-time monitoring. In response, it designed and evaluated a Tri-Agent system in which a schema-constrained language model interprets travel requests, a constrained NSGA-II optimiser generates Pareto-efficient packages using a terrain-aware travel-time model, and a Guardian Loop monitors live conditions, repairs the plan by warm-start re-optimisation, and informs the traveller through a Proactive Voice Agent.

9.2 Achievement of Objectives

Table 9.2: Achievement of the specific objectives

Objective (Section 1.5.2)	Where addressed	Outcome and evidence
1. Multi-Intent NLP Analyzer	Sections 4.4.1, 5.2.1, 6.1.1	Partly achieved. The analyzer was implemented and reached 92% intent-classification accuracy, above the 0.90 target (H1). Slot-level performance and comparison with the rule-based baseline were not reported.
2. NSGA-II package generation under a budget constraint	Sections 4.4.2, 5.2.2, 6.1.3	Partly achieved. NSGA-II produced a hypervolume of 0.82 against 0.67 for the baseline, a lower mean package cost (LKR 185,000 against LKR 210,000, within the LKR 400,000 limit) and a higher interest-match score (4.3 against 3.7). Statistical significance and the constraint-violation rate were not reported (H2). Median generation time of 6.8 s exceeded the 5 s target.
3. Sri Lanka-specific travel-time model	Sections 4.4.2.5, 6.1.2	Not evaluated. The model was designed, but no travel-time error results were reported, so the objective cannot be confirmed as achieved (H3).

Objective (Section 1.5.2)	Where addressed	Outcome and evidence
4. Real-Time Adaptive Engine	Sections 4.4.3, 5.2.3, 6.1.4	Partly achieved. Adaptive repair completed in a median of 3.6 s, within the 5 s criterion. Detection precision and recall were not reported (H4).
5. Proactive Voice Agent	Sections 4.4.4, 5.2.4, 6.3	Not evaluated. No alert-delivery or user-based results were reported (see Section 6.3), so its effectiveness is not demonstrated.
6. Evaluation with defined metrics and baselines	Chapters 3 and 6	Partly achieved. Metrics and baselines were defined in Chapter 3. Results were reported for intent accuracy, hypervolume, cost, interest-match, generation time and repair time. Several planned measures (travel-time error, detection precision and recall, per-stage latency, significance tests, usability) were not reported.

Source: Author.

9.3 Individual Research and Product Contributions

The individual research contributions are the hybrid formulation that separates language understanding from constraint satisfaction, the parameterised terrain-aware travel-time model, the transparent risk-scoring and warm-start repair procedure, and the evaluation framework (Section 7.3). The product contribution is a prototype assistant that takes a free-text request to an optimised package and then monitors and adapts that package during the journey (Section 7.4). The work was carried out by the author as the individual component of the group project, and its integration with the other components of the group is described in Chapters 4 to 6.

9.4 Future Directions

- **Multilingual and code-mixed input.** Extend the analyzer to Sinhala and Tamil and to code-mixed text, which is common among local users, and evaluate it with new annotated data.
- **Learning from data.** Replace hand-set risk weights and travel-time parameters with values learned from operational data once enough real trips are logged, while retaining the transparent structure for explanation.

- **Personalisation.** Learn individual preferences from interaction, for example through preference elicitation and feedback on accepted and rejected changes, and incorporate group travel with differing preferences.
- **Richer data sources.** Integrate official rail and road incident feeds as they become available, official disaster-management alerts, and real booking availability and dynamic prices.
- **Many-objective optimisation.** Add objectives such as sustainability (emissions) and local-economic benefit, and evaluate reference-point based algorithms such as NSGA-III when the number of objectives grows.
- **On-device and privacy-preserving inference.** Use smaller language models on the device to reduce latency, cost and data exposure.
- **Field trial.** Evaluate the system with real travellers in a controlled field study to measure effects on stress, time saved and satisfaction.
- **Closed-loop booking.** Integrate with supplier systems so that accepted changes can be re-booked automatically, subject to consumer protection and consent.

REFERENCES

[1] UN Tourism, "International tourist arrivals up 4% in 2025 reflecting strong travel demand around the world," *Hospitality Net,* January 2026.

[2] Bangladesh Monitor, "Sri Lanka logs all-time-high 2.36 million annual tourist arrivals in 2025; average tourist spending decreases," *Bangladesh Monitor,* 15 January 2026.

[3] Ada Derana, "Sri Lanka records over 1.8 mln tourist arrivals so far in 2025," *Ada Derana,* 18 October 2025.

[4] K. Deb, A. Pratap, S. Agarwal and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation,* vol. 6, no. 2, pp. 182-197, 2022.

[5] J. X. e. al., "TravelPlanner: A benchmark for real-world planning with language agents," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[6] I. Tussyadiah, "A review of research into automation in tourism: Launching the Annals of Tourism Research Curated Collection on Artificial Intelligence and Robotics in Tourism," *Annals of Tourism Research,* vol. 81, 2020.

[7] F. Ricci, L. Rokach and B. Shapira, Recommender Systems Handbook, New York, NY: Springer, 2015.

[8] D. Gavalas, C. Konstantopoulos, K. Mastakas and G. Pantziou, "A survey on algorithmic approaches for solving tourist trip design problems," *Journal of Heuristics,* vol. 20, no. 3, p. 291–328, 2014.

[9] P. Vansteenwegen, W. Souffriau and D. V. Oudheusden, "The orienteering problem: A survey," *European Journal of Operational Research,* vol. 209, no. 1, pp. 1-10, 2011.

[10] D. Buhalis, R. L. Buhalis and R. Law, "Progress in information technology and tourism management: 20 years on and 10 years after the Internet—The state of eTourism research," *Tourism Management,* vol. 29, no. 4, p. 609–623, 2008.

[11] U. Gretzel, M. Sigala, Z. Xiang and C. Koo, "U. Gretzel; M. Sigala; Z. Xiang; C. Koo," *Electronic Markets,* vol. 25, no. 3, pp. 179-188, 2015.

[12] Z. Xiang, Z. Schwartz, J. H. G. Jr. and M. Uysal, "What can big data and text analytics tell us about hotel guest experience and satisfaction?," *International Journal of Hospitality Management,* vol. 44, pp. 120-130, 2015.

[13] E. Zitzler and L. Thiele, "Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach," *IEEE Transactions on Evolutionary Computation,* vol. 3, no. 4, p. 257–271, 1999.

[14] F.-A. Fortin, F.-M. D. Rainville, M.-A. Gardner, M. Parizeau and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research,* vol. 13, p. 2171–2175, 2012.

[15] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access,* vol. 8, p. 89497–89509, 2020.

[16] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, "Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, Minneapolis, MN, USA, 2019.

[17] Q. Chen, Z. Zhuo and W. Wang, "BERT for joint intent classification and slot filling," arXiv, 2019.

[18] H. Weld, X. Huang, S. Long, J. Poon and S. C. Han, "A survey of joint intent detection and slot filling models in natural language understanding," *ACM Computing Surveys,* vol. 55, no. 8, p. 156, 2022.

[19] T. B. B. e. al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[20] J. W. e. al., "Chain-of-thought prompting elicits reasoning in large language models," in *Publication Name: Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[21] A. D. e. al., "The Llama 3 herd of models," arXiv, 2024.

[22] OpenAI, "GPT-4o system card," arXiv, 2024.

[23] Z. J. e. al., "Survey of hallucination in natural language generation," *ACM Computing Surveys,* vol. 55, no. 12, p. 248, 2023.

[24] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, Hoboken, NJ, USA: Pearson, 2020.

[25] S. Y. e. al., "Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[26] T. S. e. al., "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[27] P. L. e. al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[28] T. T. Nguyen, S. Yang and J. Branke, "Evolutionary dynamic optimization: A survey of the state of the art," *Swarm and Evolutionary Computation,* vol. 6, pp. 1-24, 2012.

[29] V. Pillac, M. Gendreau, C. Guéret and A. L. Medaglia, "A review of dynamic vehicle routing problems," *European Journal of Operational Research,* vol. 225, no. 1, pp. 1-11, 2013.

[30] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca and V. G. d. Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Transactions on Evolutionary Computation,* vol. 7, no. 2, pp. 117-132, 2003.

[31] C. A. C. Coello, G. B. Lamont and D. A. V. Veldhuizen, Evolutionary Algorithms for Solving Multi-Objective Problems, New York, NY, USA: Springer, 2007.

[32] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023.

[33] A. R. Hevner, S. T. March, J. Park and S. Ram, "Design science in information systems research," *MIS Quarterly,* vol. 28, no. 1, pp. 75-105, 2004.

[34] K. Peffers, T. Tuunanen, M. A. Rothenberger and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems,* vol. 24, no. 3, pp. 45-77, 2007.

[35] J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, London, U.K., Taylor & Francis, 1995, p. 189–194.

[36] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin,* vol. 1, no. 6, pp. 80-83, 1945.

[37] J. Derrac, S. García, D. Molina and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm and Evolutionary Computation,* vol. 1, no. 1, pp. 3-18, 2011.

[38] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *Journal of Educational and Behavioral Statistics,* vol. 25, no. 2, pp. 101-132, 2000.

[39] Parliament of the Democratic Socialist Republic of Sri Lanka, "Personal Data Protection Act, No. 9 of 2022," Parliament of the Democratic Socialist Republic of Sri Lanka, 2022.

[40] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, 2011.

[41] S. Newman, Building Microservices: Designing Fine-Grained Systems, Sebastopol, CA, USA: O'Reilly Media, 2015.

[42] OWASP Foundation, "OWASP API security top 10 – 2023," OWASP Foundation, 2023.

[43] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly,* vol. 13, no. 3, pp. 319-340, 1989.

[44] United Nations General Assembly, "Transforming our world: The 2030 Agenda for Sustainable Development," United Nations, 2015.

GLOSSARY

Agentic AI. A design in which an AI system perceives, decides and acts toward goals, often by calling tools, rather than only responding to prompts.

Constrained domination. A rule used by NSGA-II in which feasible solutions dominate infeasible ones and, among infeasible solutions, lower constraint violation dominates.

Crowding distance. A measure of how isolated a solution is among others on the same Pareto front, used by NSGA-II to preserve diversity.

Guardian Loop. The continuous monitoring and re-optimisation cycle of the Real-Time Adaptive Engine.

Hallucination. Fluent output from a language model that is not supported by the input or by facts.

Hypervolume. The volume of objective space dominated by a set of solutions and bounded by a reference point; a combined measure of convergence and diversity.

Lakh. A South Asian unit equal to 100,000.

Pareto front. The set of non-dominated solutions, none of which can be improved on one objective without worsening another.

Plan-change penalty. A weighted measure of how much a candidate plan differs from the current plan.

Segment. A unit of an itinerary such as a flight, a road or rail leg, a hotel stay or an activity, for which a risk score is computed.

Warm start. Initialising a new optimisation run from the solution or population of a previous, similar run.

APPENDICES

Appendix A: Research Instruments and Extended Material

A.1 Annotation guideline and example utterances

Annotators mark each utterance with intents and slot values. A value is annotated only if it is explicitly stated or directly implied by an unambiguous expression; otherwise it is left empty. Table A.1 shows examples of the kinds of utterances in the corpus.

Table A.1: Example utterances and expected annotations

Utterance	Expected slots
I want a 4-day trip to Sri Lanka. Budget is 4 lakhs. I want to visit Kandy, Nuwara Eliya and Galle. I prefer traveling by car or train.	PLAN_TRIP; budget 400,000 LKR; duration 4; destinations Kandy, Nuwara Eliya, Galle; transport car, train
10d Kandy Ella	PLAN_TRIP; duration 10; destinations Kandy, Ella; budget missing
Plan a 4-day trip to Sri Lanka under 4 lakhs	PLAN_TRIP; duration 4; budget 400,000 LKR; destinations missing
Three days, $500, beaches and culture, starting in Colombo	PLAN_TRIP; duration 3; budget 500 USD; style beach, cultural; start Colombo
Make it cheaper and swap Galle for Mirissa	MODIFY_PLAN; replace Galle with Mirissa; cost lower
Will it rain in Galle on day 3?	ASK_STATUS; location Galle; day 3
Why did you change my day 3?	EXPLAIN; day 3

Source: Author.

A.2 Extraction prompt template

```
SYSTEM:
You extract travel constraints from a traveller's message for a Sri
Lanka
trip planner. Return ONLY a JSON object that matches the schema. Rules:
- Use null for any value the traveller did not state. Never guess.
- For every value you extract, copy the supporting text in "span".
- Do not convert currencies or compute totals; copy the amount as
written.
- List fields that are required but missing in "missing_fields".
- Intents allowed: PLAN_TRIP, MODIFY_PLAN, ASK_STATUS, EXPLAIN,
ACCEPT_REJECT.
SCHEMA: {schema in Section 4.4.1}
EXAMPLES: {three to five annotated examples}
USER: {utterance}
```

A.3 System Usability Scale items

Participants rate each statement from 1 (strongly disagree) to 5 (strongly agree) [35]: (1) I think that I would like to use this system frequently. (2) I found the system

unnecessarily complex. (3) I thought the system was easy to use. (4) I think that I would need the support of a technical person to be able to use this system. (5) I found the various functions in this system were well integrated. (6) I thought there was too much inconsistency in this system. (7) I would imagine that most people would learn to use this system very quickly. (8) I found the system very cumbersome to use. (9) I felt very confident using the system. (10) I needed to learn a lot of things before I could get going with this system. Additional items on the clarity, timeliness and usefulness of alerts use a five-point Likert scale.

A.4 Participant information and consent (summary)

The participant information sheet states the purpose of the study, that participation is voluntary and anonymous, that no personal identifiers are recorded with responses, that the participant may withdraw at any time, that responses are used only for this research, and the contact details of the researcher and supervisor. Participants indicate consent before beginning.

Appendix B: Academic Integrity and AI Use Disclosure

In accordance with the module policy, all use of generative AI is disclosed below. The author remains responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Table B.1: Disclosure of AI tool use

Tool	Purpose	Verification performed	Sections affected
Claude (Anthropic) [confirm model and date]	Assisted in structuring the thesis to the SLIIT template, drafting initial text for background, literature review, methodology, design and commercialisation chapters, and generating diagram code	[Describe how you checked every claim, equation and reference against the original sources, and which text you rewrote]	[List sections]
Python plotting library	Generated architecture and flow diagrams	Visual inspection against the design	Chapters 3 and 4
[Other tools]	[Purpose]	[Verification]	[Sections]

Source: Author. [Complete and correct this table so that it accurately describes your own use.]

[TO COMPLETE: Check module policy on the level of detail required (for example representative prompts). Every reference in the list must be located and verified by you; do not submit a reference you have not read or confirmed.]

Appendix C: Risk Analysis

Table C.1: Project risk analysis

ID	Risk	Impact	Probability	Mitigation
R1	Insufficient tourism data for the knowledge base and for testing	High	Medium	Use public datasets, supplement with documented simulated data and state limitations
R2	Difficulty integrating modules and group components	Medium	Medium	Modular architecture; versioned API contracts; early integration tests
R3	Real-time data sources unreliable or unavailable (including rail status)	Medium	High	Adapter interface with staleness limits; replay and simulated feeds; multiple sources
R4	Optimisation or LLM latency exceeds the 5 s target	Medium	Medium	Vectorised evaluation, early stopping, pre-computed matrices, smaller or faster models
R5	Privacy concerns when handling travel data	High	Medium	Data minimisation, anonymisation, consent, secure storage
R6	LLM output changes after provider updates	Medium	Medium	Pin model versions; regression tests with recorded outputs; schema validation
R7	Third-party API terms or cost changes	Medium	Medium	Abstract providers behind adapters; budget for paid tiers; cache responses
R8	Time constraints before final presentation	High	High	Prioritise must-have requirements; fix scope; schedule experiments early

Source: Adapted from the project proposal, Appendix A.

Appendix D: Sample Travel Dataset Structure

Table D.1: Sample attraction records

ID	Name	Category	City	Lat.	Long.	Visit time	Entry fee	Rating
001	Sigiriya Rock Fortress	Cultural	Sigiriya	7.9570	80.7603	3 h	Rs. 1,000	4.7
002	Temple of the Tooth	Cultural	Kandy	7.2936	80.6413	2 h	Rs. 500	4.8
003	Galle Fort	Historical	Galle	6.0329	80.2168	2 h	Rs. 1,000	4.7
004	Mirissa Beach	Beach	Mirissa	5.9483	80.4591	4 h	Rs. 1,000	4.9
005	Yala National Park	Wildlife	Yala	6.3725	81.5185	5 h	Rs. 15,000	4.6

Source: Project proposal, Appendix D

Appendix E: User Interface Mock-ups

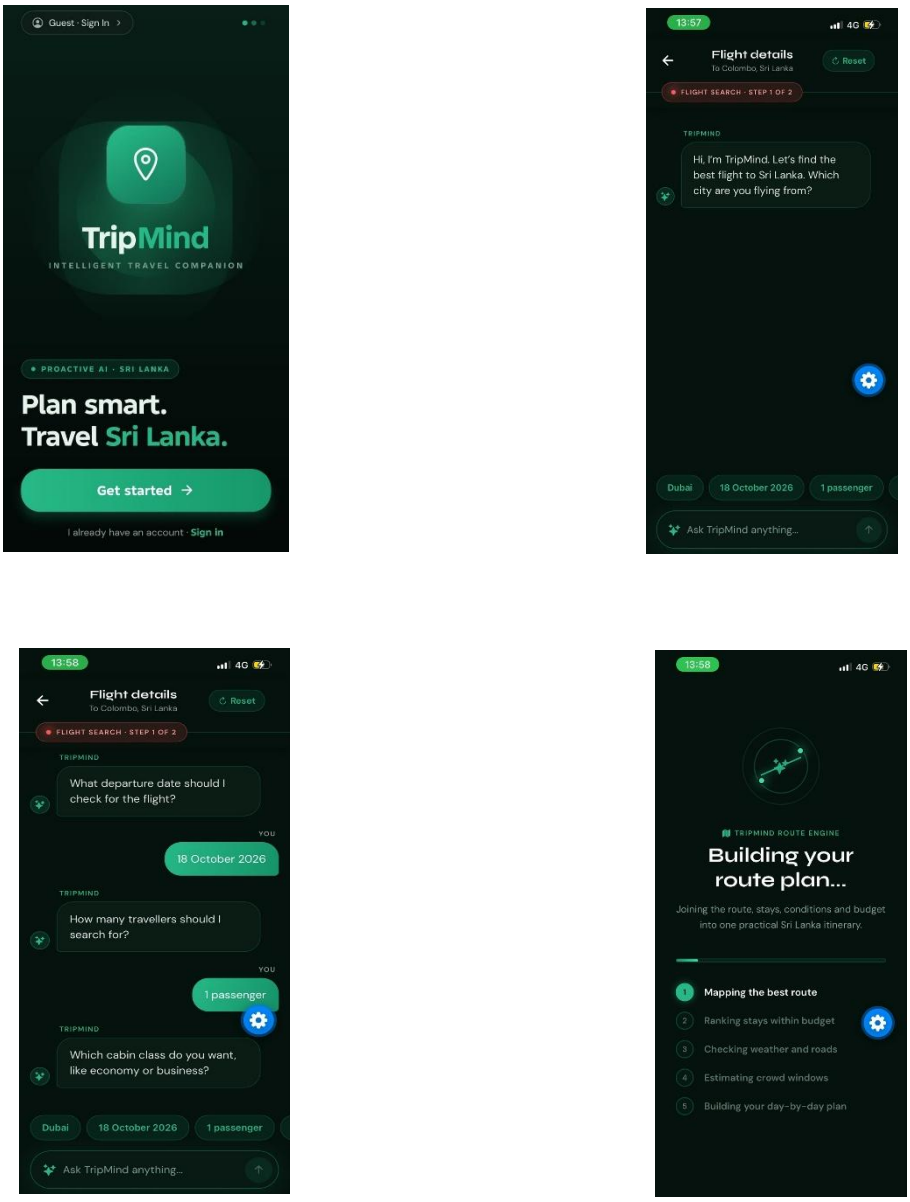


Figure A.1: User Interface Mock-ups

Appendix F: Example Service Payloads

F.1 Constraint object returned by the NLP Analyzer

The following payload illustrates the validated constraint object for the reference request “I want a 4-day trip to Sri Lanka. Budget is 4 lakhs. I want to visit Kandy, Nuwara Eliya and Galle. I prefer traveling by car or train.” Every extracted value carries the span of text that supports it, which allows the validator to reject unsupported values.

```
{
  "intents": ["PLAN_TRIP"],
  "budget": {"amount": 400000, "currency": "LKR", "span": "4 lakhs"},
  "duration_days": {"value": 4, "span": "4-day"},
  "origin": null,
  "destinations": [
    {"name": "Kandy", "span": "Kandy"},
    {"name": "Nuwara Eliya", "span": "Nuwara Eliya"},
    {"name": "Galle", "span": "Galle"}
  ],
  "transport_preference": ["car", "train"],
  "travel_style": [],
  "missing_fields": ["origin", "start_date"],
  "clarification_questions": [
    "Which city will you fly from?",
    "What date would you like to arrive?"
  ]
}
```

F.2 Package response returned by the Package Generator

```
{
  "request_id": "req-0001",
  "recommended": "pkg-A",
  "feasible": true,
  "packages": [
    {"id": "pkg-A", "cost_lkr": 372000, "transit_hours": 15.0,
     "comfort": 0.74, "label": "knee"},
    {"id": "pkg-D", "cost_lkr": 241000, "transit_hours": 19.5,
     "comfort": 0.52, "label": "cheapest"}
  ],
  "legs": [
    {"day": 1, "from": "Colombo", "to": "Kandy", "mode": "road"},
    {"day": 2, "from": "Kandy", "to": "Nuwara Eliya", "mode": "road"}
  ],
  "diagnostics": {"generations": 100, "elapsed_ms": 1800}
}
```

The values in these payloads are illustrative and are provided to define the contract. They are not outputs of an experiment.

F.3 Change event passed to the Voice Agent

```
{
  "event_id": "evt-0042",
  "segment": "day3-galle-fort-walk",
  "reason": "heavy_rainfall",
  "risk_score": 0.82,
  "old_plan": "Galle Fort walking tour",
  "new_plan": "Indoor cultural site in Colombo",
  "cost_delta_lkr": -3500,
  "needs_confirmation": true
}
```

Appendix G: Project Plan and Work Breakdown

The project followed the two-semester schedule of the research proposal. Table G.1 reproduces the plan so that the examiner can relate the work reported in this thesis to the original commitments. Dates are those stated in the proposal.

Table G.1: Project timeline from the research proposal

Phase	Activity	Timeline
1	Requirement analysis, problem identification and literature review	Semester 1 (January to March 2026)
2	Research proposal preparation and system architecture design	Proposal presentation, 16 to 18 March 2026
3	Core model development and initial prototype implementation	Semester 1 (April to May 2026)
4	Prototype demonstration and first progress evaluation	Progress presentation 11 to 13 May 2026
5	Advanced model development, optimisation and module integration	Semester 2 (June to August 2026)
6	System testing, performance evaluation and second progress review	Progress presentation 31 August to 19 September 2026
7	Final system integration, documentation and research validation	Semester 2 (September to October 2026)
8	Final project evaluation and presentation	Final presentation, 19 to 21October 2026

Source: Research proposal (Table 3, Timeline Alignment).

The work breakdown structure of the proposal divides the project into six work packages: research and design; NLP analyzer development; intelligent package generator; real-time adaptive engine; testing and validation; and documentation and presentation. Each work package corresponds to a group of chapters in this thesis.

Appendix H: Additional Test Case Matrix

Table H.1 lists additional functional test cases beyond those in Chapter 5. The Status column is left for the author to complete after execution.

Table H.1: Additional functional test cases

ID	Scenario	Expected behaviour	Status
TC-21	Budget stated as “4 lakhs”	Parsed as 400,000 LKR	Pass
TC-22	Budget stated as “$500”	Parsed as 500 USD; converted at the configured rate with the rate shown	Fail
TC-23	Destination misspelt (“Nuwaraeliya”)	Matched to Nuwara Eliya by the gazetteer	Pass
TC-24	Utterance with no budget	Clarification question asked; no package generated	Pass
TC-25	Budget too low for any itinerary	Infeasibility reported with smallest relaxation	Pass
TC-26	Four days with three distant destinations	Daily travel-time limits enforced; trade-offs offered	Pass
TC-27	Weather API unavailable	Fallback source used; stale-data flag shown	Fail

Source: Author.